Diplomado en Arquitectura de Software

Módulo 2 - SOLID & Patrones de diseño

Prof. César Augusto Vega Fernández

Facultad de **Ingeniería**

Universidad de La Sabana

VIGILADA MINEDUCACIÓN

Universidad de La Sabana | Facultad de **Ingeniería**



Bienvenidos

En este módulo, nos adentraremos en el mundo de **SOLID**, un conjunto de principios diseñados para guiar el diseño de software hacia la creación de componentes independientes, fáciles de mantener y de ampliar. Además, exploraremos una variedad de **patrones de diseño**, desde los creacionales, que nos permiten instanciar objetos de manera flexible, hasta los estructurales y de comportamiento, que nos ayudan a organizar y coordinar nuestras clases de manera efectiva.

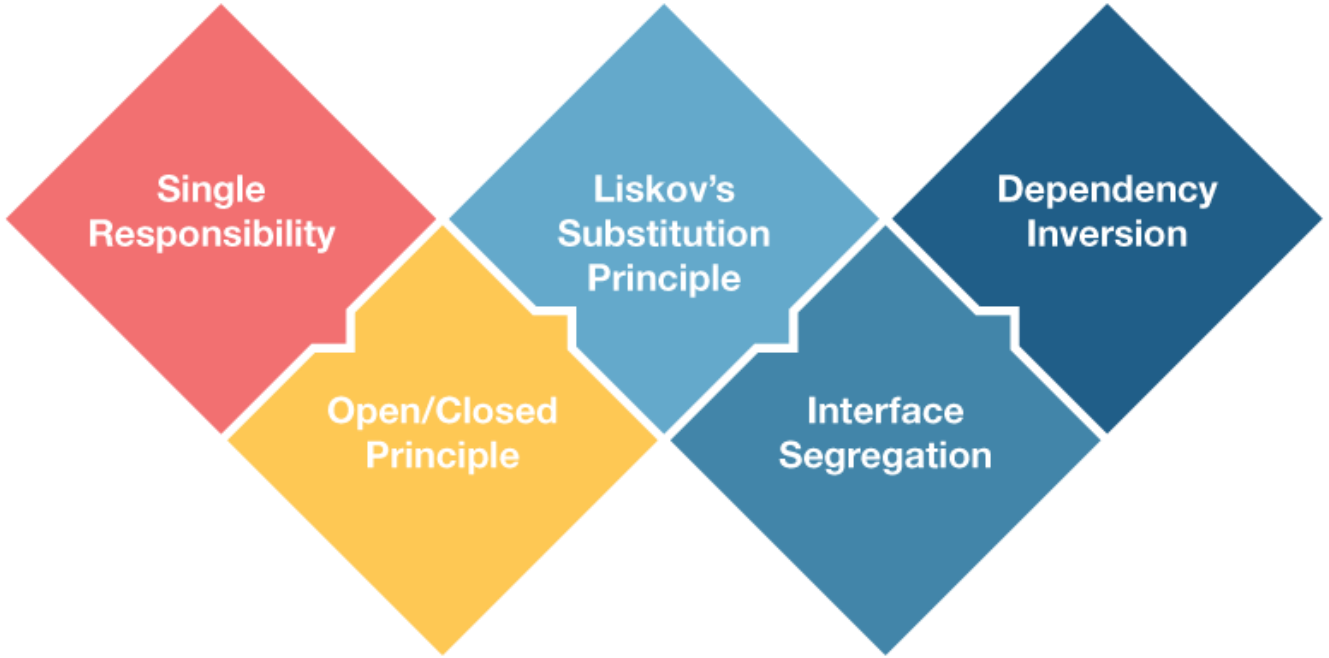


¿Qué son los principios SOLID?

Los principios SOLID son guías que pueden ser aplicadas en el desarrollo de software para eliminar malos diseños provocando que el programador tenga que refactorizar el código fuente hasta que sea legible y extensible.


Edificio Ad Portas

S.O.L.I.D.


Single Responsibility
Liskov's Substitution Principle
Dependency Inversion
Open/Closed Principle
Interface Segregation

Single Responsibility Principle (SRP)

Definición:

- ❑ Un objeto debe tener una razón para cambiar, es decir, debe tener una única responsabilidad.

Sugerencia:

- ❑ Este principio sugiere que una clase o módulo debe tener solo una razón para cambiar, lo que implica una única responsabilidad en el sistema.

Ejemplos Prácticos:

- ❑ Una clase de gestión de empleados debería tener la responsabilidad de gestionar empleados, pero no la de generar informes financieros.



Facultad de **Ingeniería**

Universidad de La Sabana

Open/Closed Principle (OCP)

Definición:

- ❑ Las entidades (clases, módulos, funciones) deben estar abiertas para su extensión, pero cerradas para su modificación.

Sugerencia:

- ❑ Esto sugiere que el código debe ser extensible para admitir nuevas funcionalidades sin cambiar el código existente.

Ejemplos Prácticos:

- ❑ Uso de interfaces y clases abstractas para permitir extensiones.



Facultad de
Ingeniería

Universidad de
La Sabana

Liskov Subtitution Principle (LSP)

Escenario:

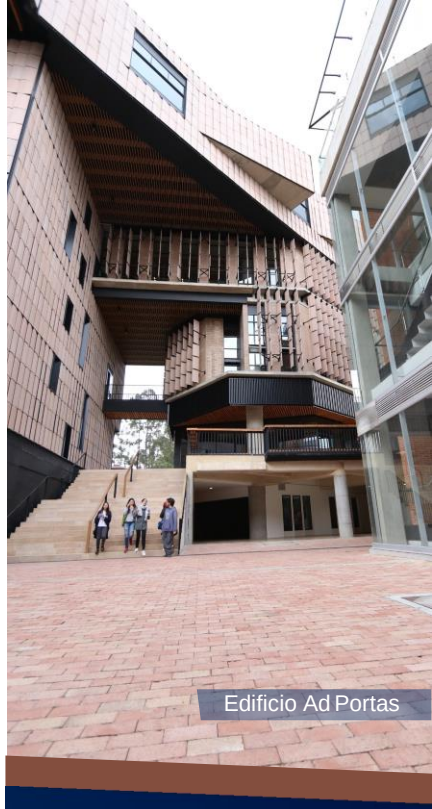
- Las instancias de una clase base deben poder ser reemplazadas por instancias de sus clases derivadas sin afectar la integridad del programa.

Sugerencia:

- Este principio garantiza la consistencia del sistema cuando se utilizan herencias y polimorfismo.

Ejemplo Real:

- Si una clase base es **Vehículo**, una clase derivada como **Coche** debería poder ser usada en lugar de **Vehículo** sin cambiar la funcionalidad.



Facultad de
Ingeniería

Universidad de
La Sabana

Interface Segregation Principle (ISP)

Escenario:

- ❑ Un cliente no debe verse obligado a depender de interfaces que no utiliza.

Sugerencia:

- ❑ Si una interfaz es demasiado grande y una clase implementa solo una parte de ella, el cliente debe depender solo de la parte que necesita.

Ejemplo Real:

- ❑ Si una clase base es **Vehículo**, una clase derivada como **Coche** debería poder no debería poder usar un método para **volar**.



Facultad de
Ingeniería

Universidad de
La Sabana

Dependency Inversion Principle (DIP)

Escenario:

- Los módulos de alto nivel no deben depender de los módulos de bajo nivel. Ambos deben depender de abstracciones.

Sugerencia:

- Las abstracciones no deben depender de detalles. Los detalles deben depender de abstracciones.

Ejemplo Real:

- Un concesionario no puede tener muchos tipos de vehículos solo debe tener vehículos



Facultad de **Ingeniería**

Universidad de La Sabana

Diseño Orientado a Objetos

El Diseño Orientado a Objetos (DOO) es un paradigma de desarrollo de software que se basa en el concepto fundamental de “objetos”. Estos objetos representan entidades del mundo real y encapsulan tanto los datos (atributos) como las operaciones (métodos) que pueden realizar.





Importancia

Acoplamiento

Cohesión

Acoplamiento

Se refiere al grado de dependencia entre los módulos o componentes de un sistema. Un bajo acoplamiento indica que los módulos son independientes y pueden cambiar sin afectar otros, mientras que un alto acoplamiento significa una fuerte interdependencia.



Facultad de
Ingeniería

Universidad de
La Sabana



Tipos de acoplamiento

Estructural

Control

Datos



Importancia de un bajo acoplamiento

- **Flexibilidad**
- **Reusabilidad**
- **Mantenimiento sencillo**

Cohesión

Se refiere al grado en que los elementos dentro de un módulo están relacionados entre sí. Un alto nivel de cohesión significa que las responsabilidades y funciones dentro de un módulo están estrechamente relacionadas y contribuyen a un propósito común.



Facultad de
Ingeniería

Universidad de
La Sabana



Tipos de cohesión

Funcional/tarea única

Comunicacional/ misma información

Secuencial

Procedural

Temporal

Importancia de una alta cohesión

Entendimiento

Reusabilidad

Modularidad



Principios básicos

Relación entre Acoplamiento y Cohesión:

- ✓ **Principio de Diseño:**
  - ✓ Buscar un equilibrio entre bajo acoplamiento y alta cohesión.
- ✓ **Consecuencias:**
  - ✓ Bajo acoplamiento y alta cohesión contribuyen a un diseño más robusto y fácil de mantener.
  - ✓ Un alto acoplamiento y baja cohesión pueden resultar en sistemas difíciles de entender y modificar.

Estrategias para Mejorar Acoplamiento y Cohesión:

- ✓ **División de Responsabilidades:** Asignar responsabilidades específicas a cada módulo.
- ✓ **Uso de Interfaces:** Definir interfaces claras para limitar las dependencias.
- ✓ **Desacoplar Funcionalidades:** Separar funciones independientes en módulos distintos.
- ✓ **Refactorización:** Reorganizar el código para mejorar la estructura y relación entre módulos.





Conclusiones SOLID

- ✓ **Flexibilidad y Adaptabilidad:**
La aplicación de los principios SOLID proporciona flexibilidad para adaptarse a cambios en los requisitos del software sin afectar otras partes del sistema.

- ✓ **Reusabilidad de Código:**
La adhesión a SOLID fomenta la reutilización de código al crear clases y módulos independientes que pueden integrarse en diferentes contextos.

- ✓ **Mantenimiento Simplificado:**
Los sistemas diseñados con principios SOLID son más fáciles de mantener, ya que las modificaciones en una parte del código no generan efectos secundarios inesperados.

- ✓ **Claridad en el Diseño:**
SOLID promueve la claridad y comprensión del diseño al organizar el código de manera coherente y alineada con los principios fundamentales de la ingeniería de software.

- ✓ **Escalabilidad:**
La aplicación de SOLID facilita la escalabilidad del software, permitiendo la incorporación de nuevas funcionalidades sin afectar la estabilidad del sistema existente.



Conclusiones DOO

- ✓ **Robustez y Mantenimiento:**
Un bajo acoplamiento y una alta cohesión contribuyen a la robustez del sistema, simplificando el mantenimiento y la corrección de errores.

- ✓ **Entendimiento del Código:**
Un diseño con alta cohesión facilita el entendimiento del código, ya que las funciones relacionadas están agrupadas de manera lógica.

- ✓ **Reutilización Efectiva:**
Bajo acoplamiento favorece la reutilización efectiva de módulos, ya que cada componente es independiente y puede ser integrado en diversos contextos.

- ✓ **Facilita la Evolución del Software:**
Un bajo acoplamiento y una alta cohesión permiten que el software evolucione de manera más sencilla a medida que se introducen nuevas características o se realizan mejoras.

- ✓ **Eficiencia en el Desarrollo:**
La atención a mantener un acoplamiento adecuado y una cohesión fuerte desde el principio del desarrollo contribuye a la eficiencia en la creación y evolución del software.



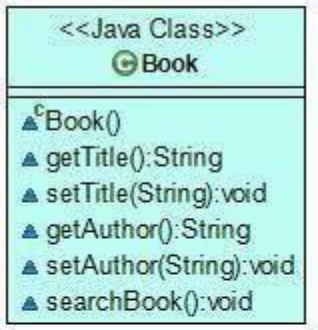
Actividad

Dado los siguientes diagramas:

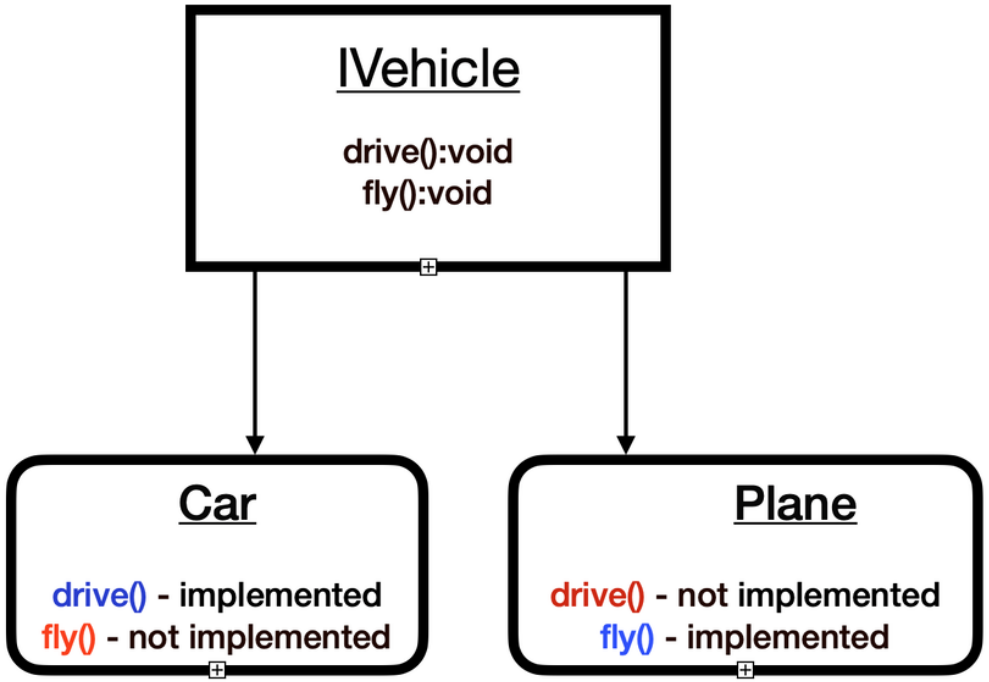
1. Decir que principio no está siguiendo
2. ¿Cómo solucionarlo?



Problema 1

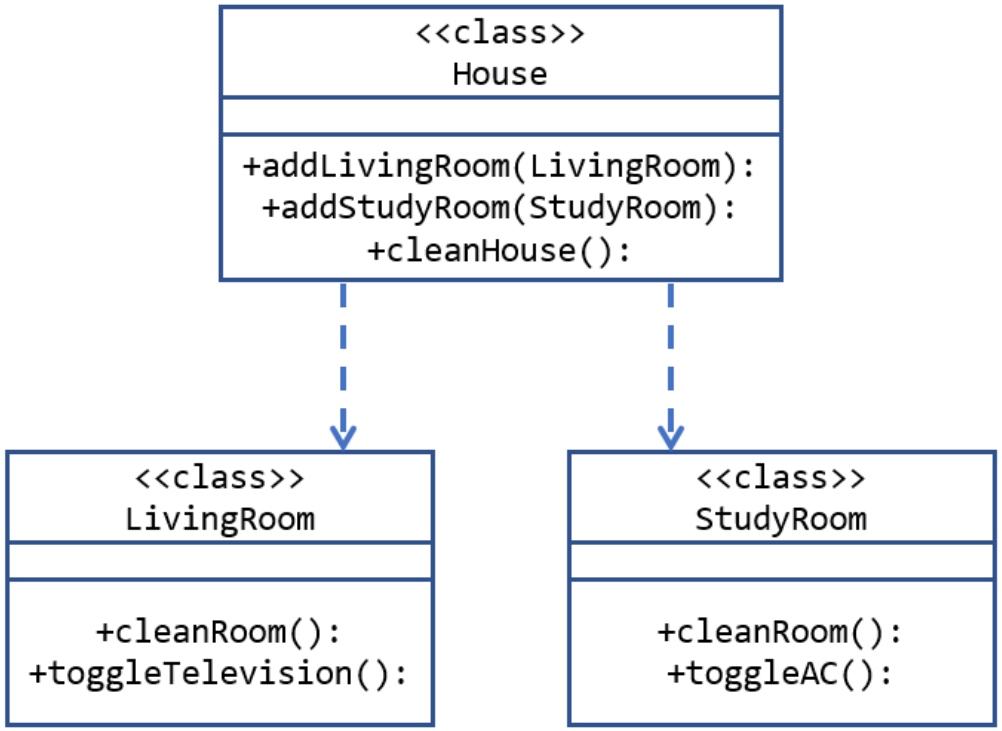


Problema 2

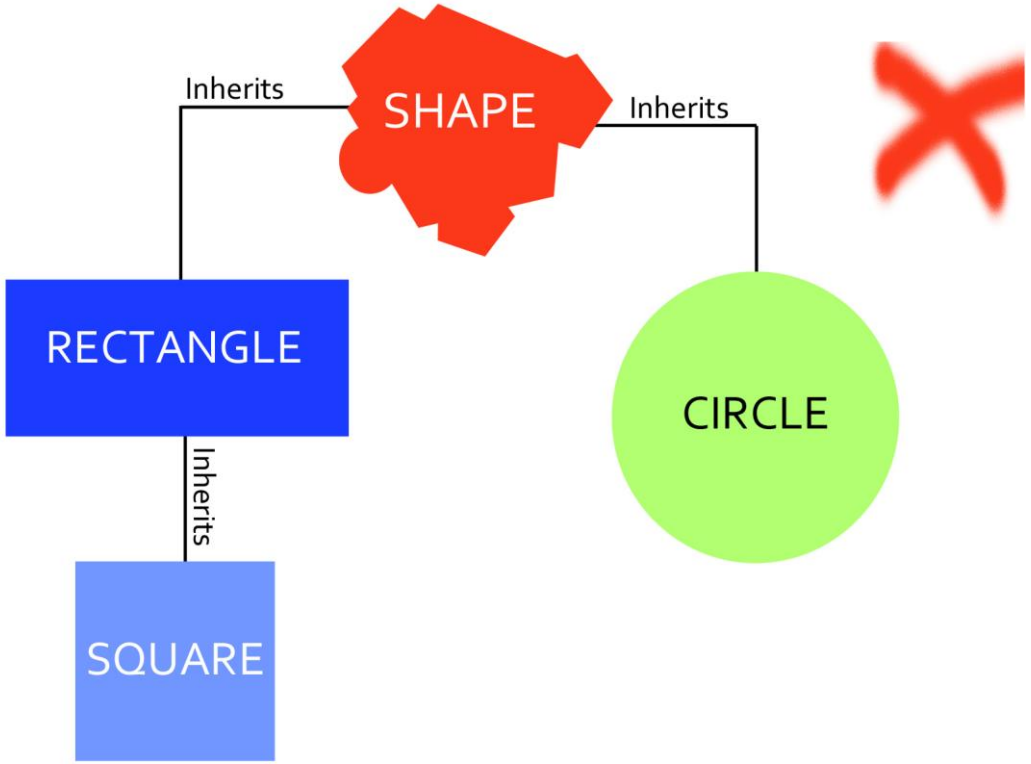




Problema 3



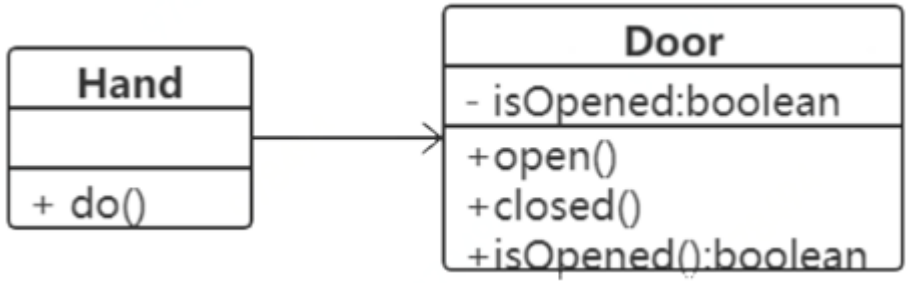
Problema 4



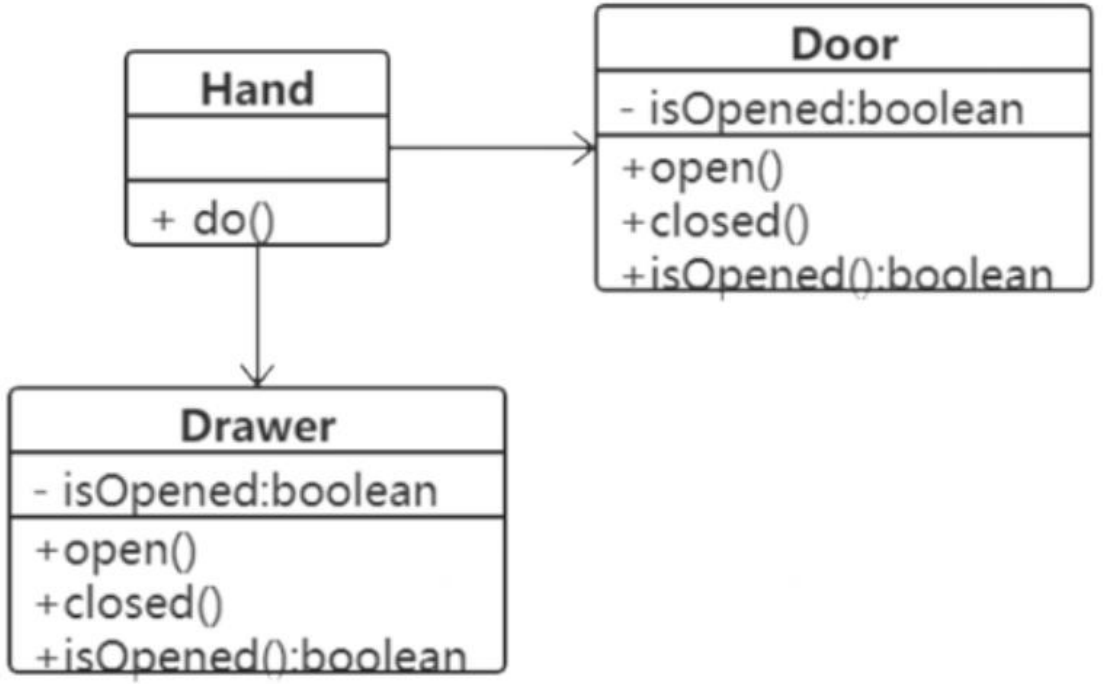


Problema 5

Un ejemplo, abrir y cerrar puertas a mano.



Si hay una nueva demanda, es necesario aumentar la función de apertura y cierre del cajón manualmente. Para el código, debe agregar una clase Cajón y modificar la clase Mano al mismo tiempo.





¿Qué son los patrones de diseño?

Un patrón de diseño es una solución general reutilizable para un problema recurrente en el diseño de software.

Funciones Principales:

- Facilitan la reutilización de soluciones exitosas.
- Ofrecen consistencia y estandarización en el diseño.

Importancia:

- Mejoran la eficiencia y mantenibilidad del código.



Categorías

Patrones Creacionales:

Se centran en la creación de objetos.

Patrones Estructurales:

Gestionan la composición de clases y objetos.

Patrones de Comportamiento:

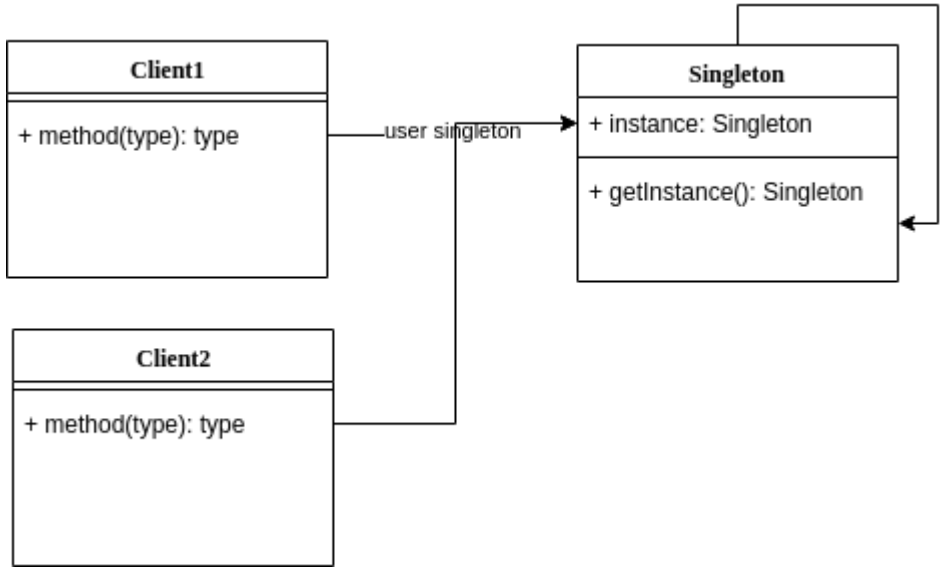
Definen la comunicación entre objetos.

Patrones Creacionales:

Singleton

Definición:

El patrón de diseño Singleton garantiza que una clase tenga solo una instancia y proporciona un punto de acceso global a esa instancia. En otras palabras, asegura que una clase tenga una única instancia y proporciona un medio para acceder a esa instancia desde cualquier punto del programa.

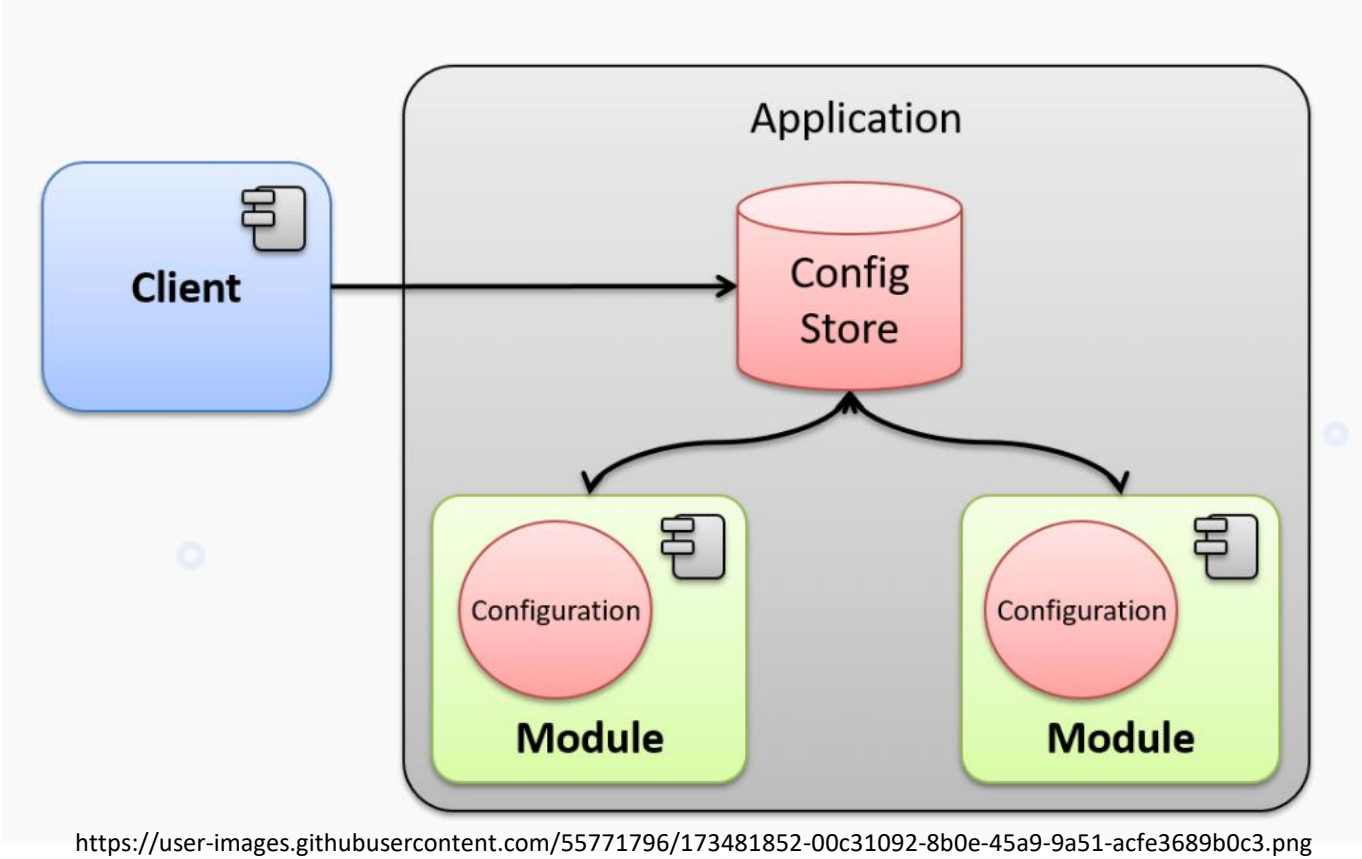




Facultad de
Ingeniería

Universidad de
La Sabana

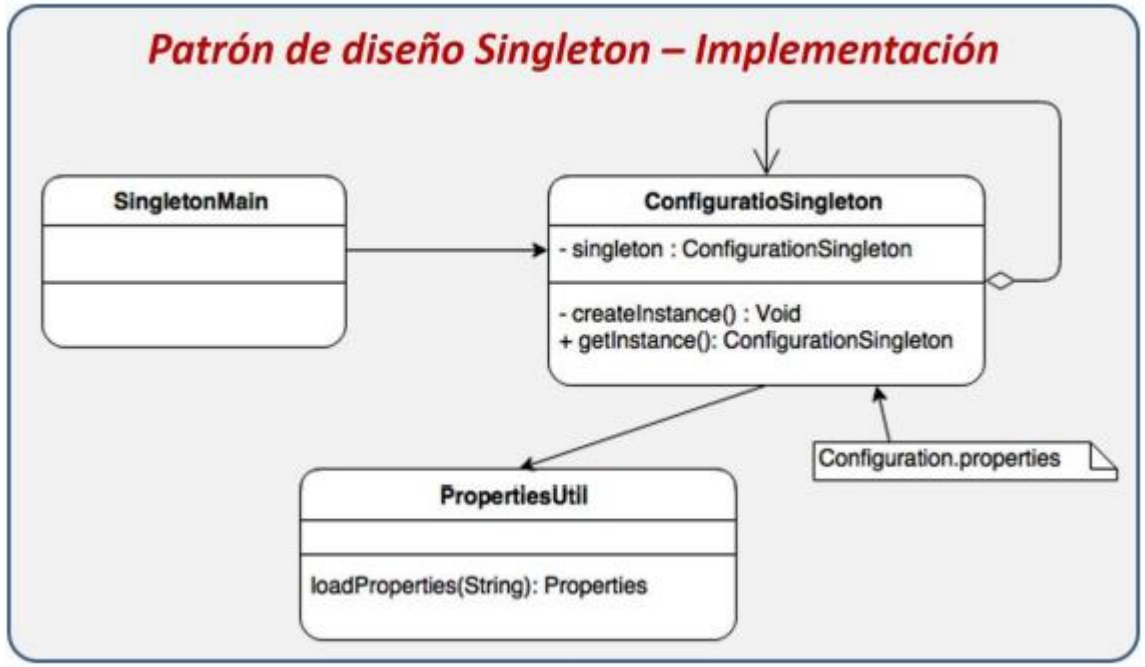
Ejemplo



https://user-images.githubusercontent.com/55771796/173481852-00c31092-8b0e-45a9-9a51-acfe3689b0c3.png



Ejemplo



https://user-images.githubusercontent.com/55771796/174157215-7aa773e1-2391-4c41-9222-79309103cebd.png



Ventajas de Singleton:

1. Control de Instancia Única: Evita la creación de múltiples instancias de la clase, garantizando la existencia de una única instancia.



2. Acceso Global: Permite el acceso global a la instancia única, facilitando la comunicación entre diferentes partes del programa.

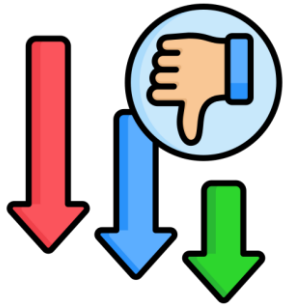
3. Eficiencia de Recursos: Útil cuando se desea evitar la sobrecarga de crear instancias múltiples de una clase, especialmente en situaciones donde la creación y destrucción de objetos es costosa.



Desventajas de Singleton:

1. **Acoplamiento Fuerte:** Puede introducir un acoplamiento fuerte en el código, ya que el Singleton se vuelve accesible globalmente.

2. Dificultad en Pruebas Unitarias: Puede complicar las pruebas unitarias, ya que el Singleton introduce un estado global que puede afectar los resultados de las pruebas.



3. Patrón a Menudo Mal Utilizado: A veces se utiliza innecesariamente, lo que puede conducir a un diseño defectuoso.



Uso del Singleton:

```
public class Main {
    public static void main(String[] args) {
        // Acceder a la instancia única de Singleton
        Singleton singletonInstance = Singleton.getInstance();

        // Utilizar métodos de la instancia única
        singletonInstance.showMessage();
    }
}
```

Ejemplo de Implementación en Java:

```
public class Singleton {
    // La instancia única de la clase
    private static Singleton instance;

    // Constructor privado para evitar la creación de instancias externas
    private Singleton() {}

    // Método para obtener la instancia única
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }

    // Métodos adicionales de la clase
    public void showMessage() {
        System.out.println("¡Hola, soy una instancia única de Singleton!");
    }
}
```




Actividad en clase

Enunciado del Problema: Integración de Singleton en una Empresa

La empresa XYZ ha decidido implementar un sistema que utilice el patrón de diseño Singleton para gestionar la impresora central y la información de los empleados en su sucursal. Se ha proporcionado un código inicial que utiliza el patrón Singleton para la impresora y un modelo básico para la clase **Empleado**. Tu tarea es completar la integración y mejorar la salida del sistema.

1. **Patrón Singleton para la Impresora:**
   - Completa la clase **Impresora** utilizando el patrón Singleton. Asegúrate de que solo haya una instancia de la impresora en todo el programa.
   - Modifica el método **print** de la impresora para imprimir el número total de páginas impresas hoy.

1. **Clase Empleado:**
   - Completa la clase **Empleado** para que utilice la instancia única de la impresora.
   - Implementa el método **printCurrentFuncion** en la clase **Empleado** que utiliza la impresora para imprimir la información del empleado, incluyendo nombre, posición y función.

3. **Integración en Main:**
   - En la clase **Main**, crea instancias de varios empleados con diferentes roles y funciones en la sucursal.
   - Llama al método **printCurrentFuncion** para cada empleado, lo que debería imprimir su información utilizando la impresora única.

4. **Salida Mejorada:**
   - Mejora el formato de salida para que la información de cada empleado se presente claramente y estéticamente.
   - Asegúrate de que la salida incluya el número total de páginas impresas hoy.

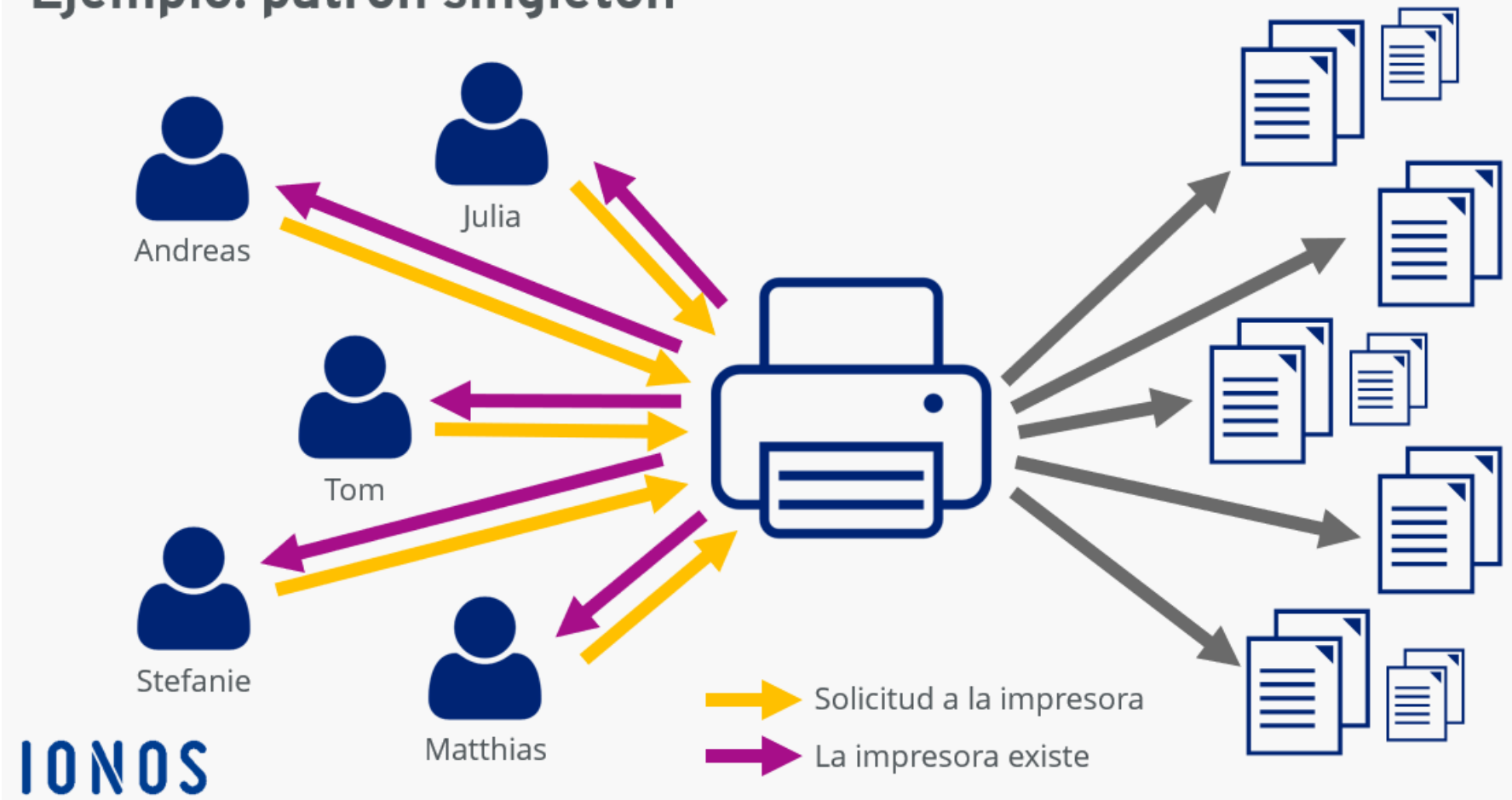


Instrucciones:

- Por grupos deben diseñar la clase **Impresora** y **Empleado** según las indicaciones.
- En el método **main** de la clase **Main**, creará instancias de empleados y llamará al método **printCurrentFuncion** para imprimir su información.
- La salida del programa debe ser clara y comprensible, incluyendo tanto la información del empleado como el total de páginas impresas hoy.



Ejemplo: patrón singleton





```
public class Main {
        public static void main(String[] args) {
                Empleado andreas = new empleado ("Andreas",
                                "Jefe",
                                "Gestiona la sucursal");
                Empleado julia = new empleado ("Julia",
                                "Consultor",
                                "Asesora a los clientes sobre las quejas");
                Empleado tom = new empleado ("Tom",
                                "Venta",
                                "Vende los productos");
                Empleado stefanie = new empleado ("Stefanie",
                                "Desarrollador",
                                " Mantenimiento informático en la sucursal.");
                Empleado matthias = new empleado ("Matthias",
                                "Contable",
                                "contabilidad financiera de la sucursal.");
                andreas.printCurrentFunción ();
                julia.printCurrentFunción ();
                tom.printCurrentFunción ();
                stefanie.printCurrentFunción ();
                matthias.printCurrentFunción ();
        }
}
```




```
public class Employee {
        private final String name;
        private final String position;
        private final String role;
        public empleado(String name, String position, String role) {
                this.nombre = nombre;
                this.posición = posición;
                this.función = función;
        }
        public void printCurrent función (){
                impresora = Impresora.getInstance();
                impresora.print("empleado: " + nombre + "\n" +
                        "Posición: " + posición + "\n" +
                        " Función: " + función + "\n");
        }
}
```




```
public class impresora {
        private static Impresora impresora;
        private int NúmeroPáginas;
        private impresora() {
        }
        public static Impresora getInstance() {
                return impresora == Null ?
                                impresora = new impresora() :
                                impresora;
        }
        public void print(String text){
                System.out.println(text +
                                "\n" + "número de páginas impresas hoy" + ++ NumeroPáginas +
                                "\n" + "---------");
        }
}
```

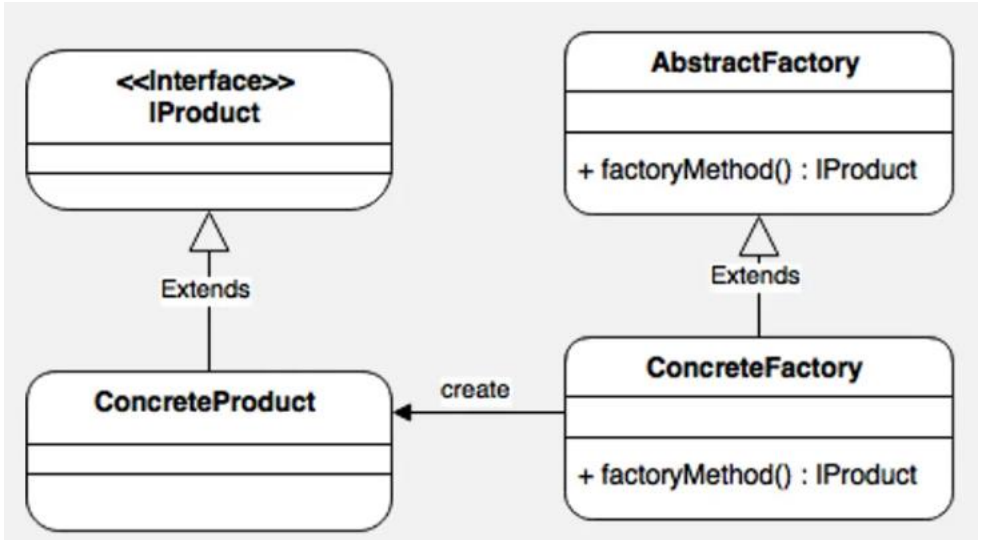



Patrones Creacionales:

Factory Method

Definición:

El Factory Method es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, pero permite a las subclases alterar el tipo de objetos que se crearán.

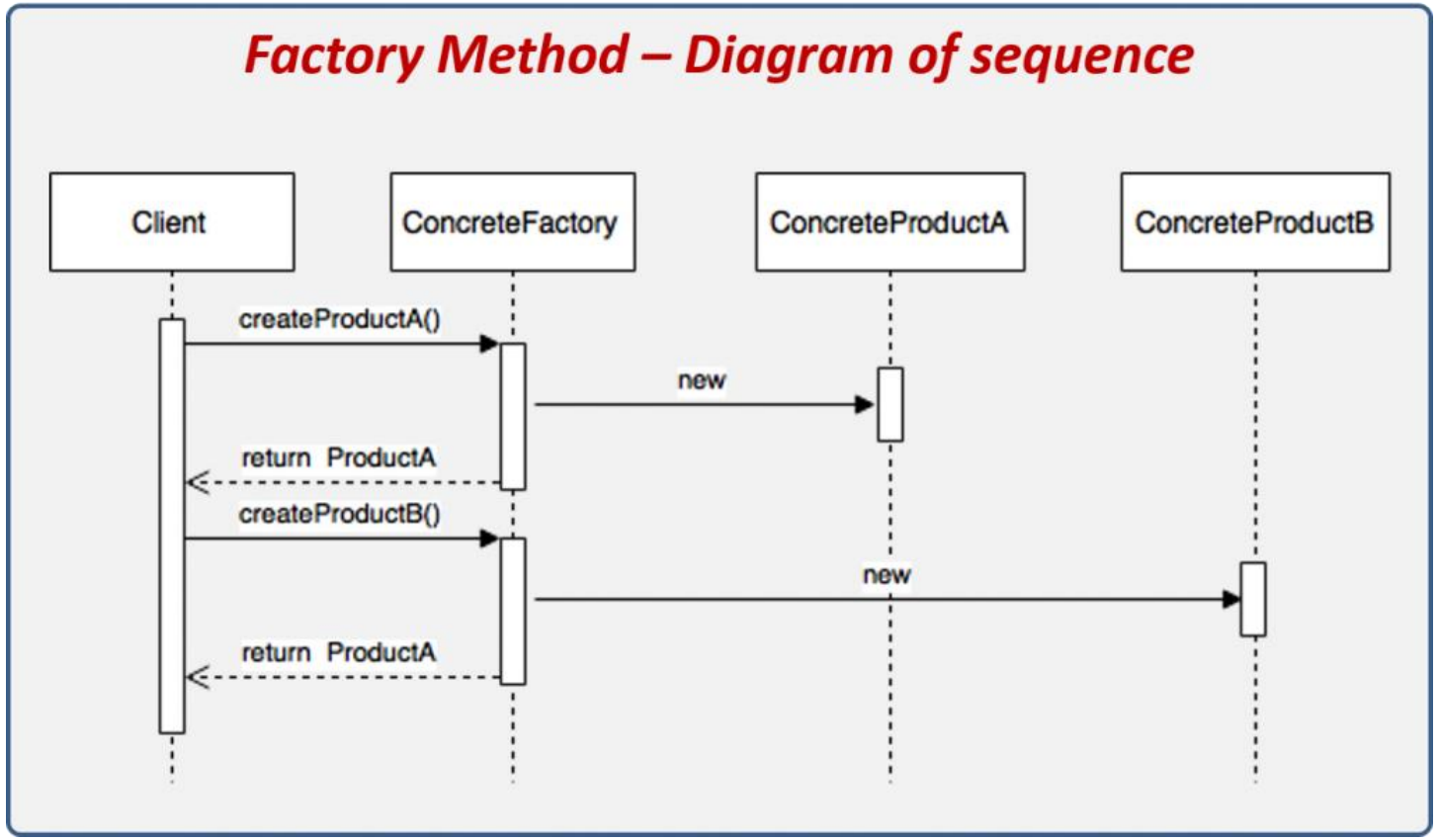




Facultad de
Ingeniería

Universidad de
La Sabana

Flujo Método de la Fabrica



https://www.oscarblancarteblog.com/2014/07/18/patron-de-diseno-factory/



Objetivo de Factory Method:

El principal objetivo del Factory Method es delegar la responsabilidad de instanciación a las subclases, permitiendo así que una clase base (superclase) proporcione una interfaz para crear objetos, pero posibilitando que las subclases tomen decisiones sobre qué clase concreta instanciar.



Ventajas de Factory Method:

Flexibilidad en la creación de objetos



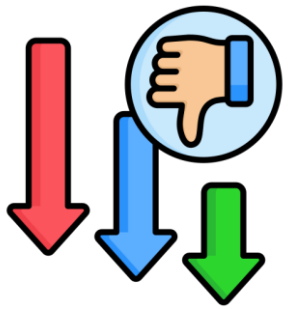
Soporte para principios SOLID, especialmente el Principio de Abierto/Cerrado.

Desacoplamiento entre la creación de objetos y su uso.



Desventajas de Factory Method:

- Complejidad Adicional
- Dificultad para Elegir entre Factory Methods
- Introducción de Muchas Interfaces o Clases Abstractas

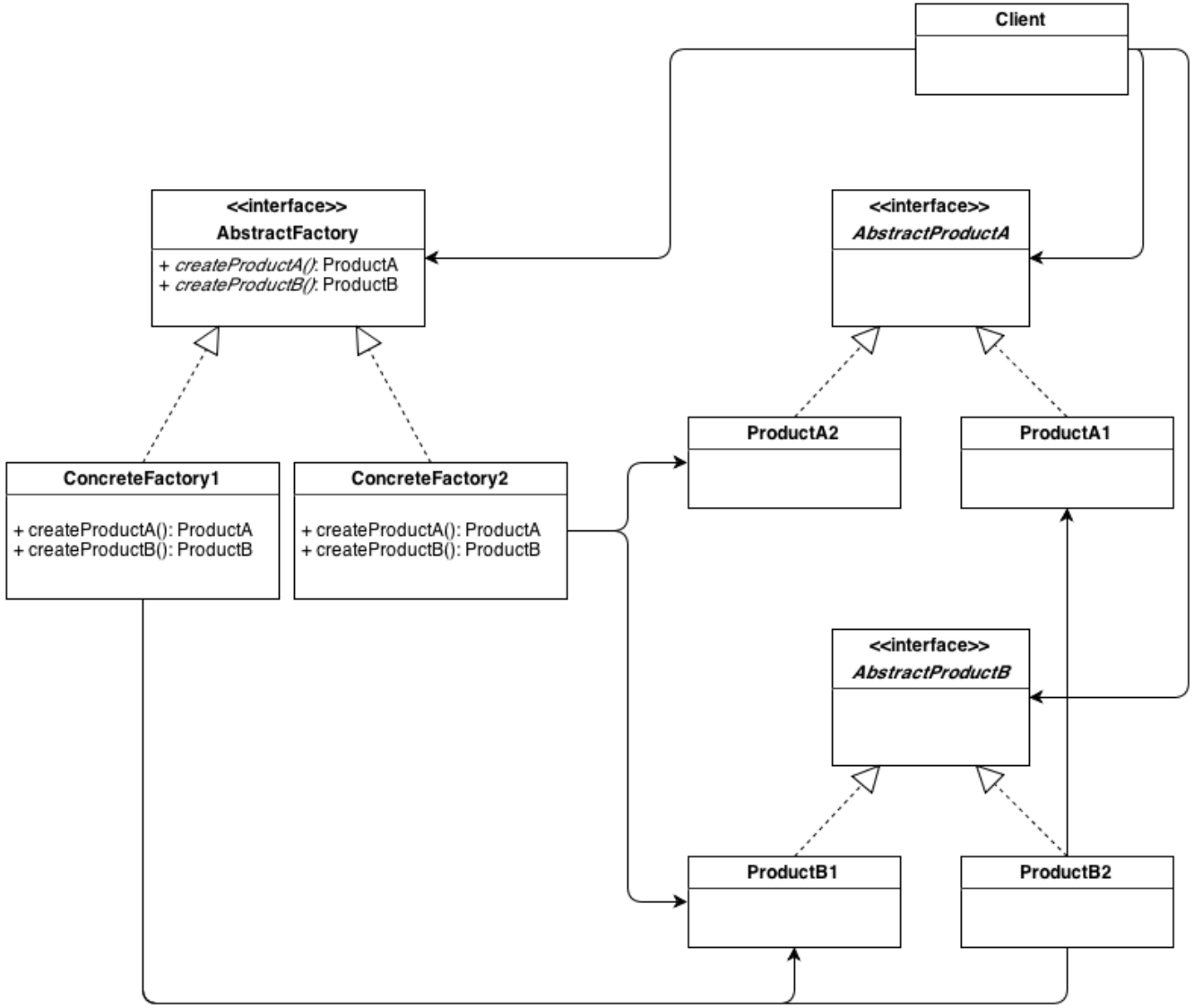


- Duplicación de Código
- Inflexibilidad con Tipos de Productos
- Dificultad en la Configuración



Ejemplo

Supongamos que cada fabrica puede crear más de un producto propio o en concreto





Uso del Factory Method:

```
public class Main {
    public static void main(String[] args) {
        FactoryMethod factory = new ConcreteFactory();
        Product product = factory.createProduct();
        product.operacion();
    }
}
```

Ejemplo de Implementación en Java:

```
// Interfaz para el producto
interface Product {
  void operacion();
}

// Interfaz o clase abstracta para el Factory Method
interface FactoryMethod {
  Product createProduct();
}

// Clase concreta que implementa el Factory Method
class ConcreteFactory implements FactoryMethod {
  @Override
   public Product createProduct() {
      return new ConcreteProduct();
   }
}

// Clase concreta del producto
class ConcreteProduct implements Product {
  @Override
  public void operacion() {
    System.out.println("Operación en ConcreteProduct");
  }
}
```

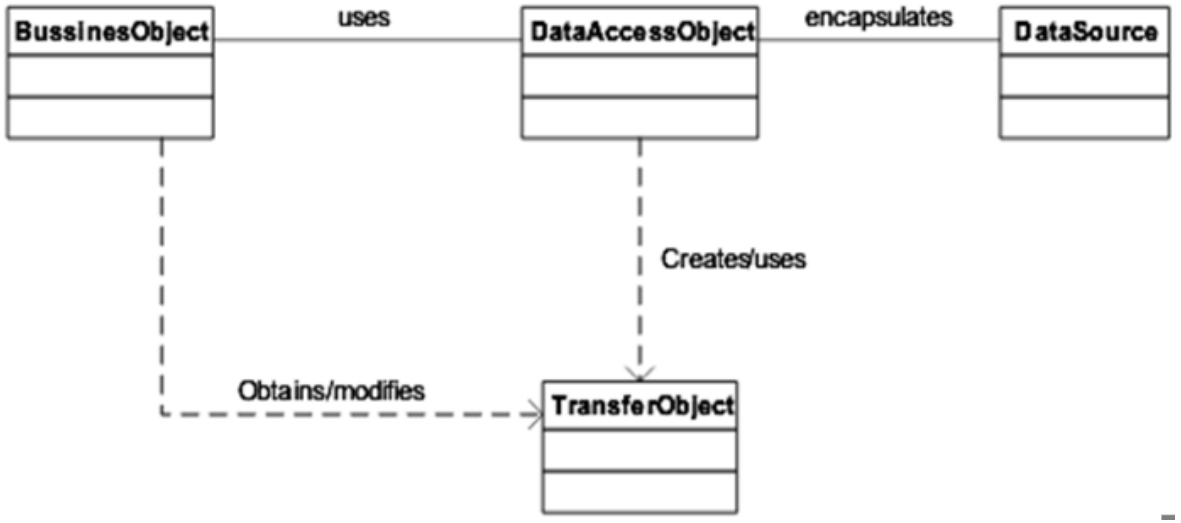



Patrones estructurales:

Patrón DAO

Definición:

El patrón DAO (Data Access Object) es un patrón de diseño que se utiliza comúnmente en el desarrollo de software para separar la lógica de acceso a datos de la lógica de negocio. Este patrón proporciona una capa de abstracción entre la aplicación y la fuente de datos, lo que facilita el intercambio de datos entre la aplicación y diferentes fuentes de datos (como una base de datos, un servicio web, etc.) sin necesidad de modificar la lógica de negocio.

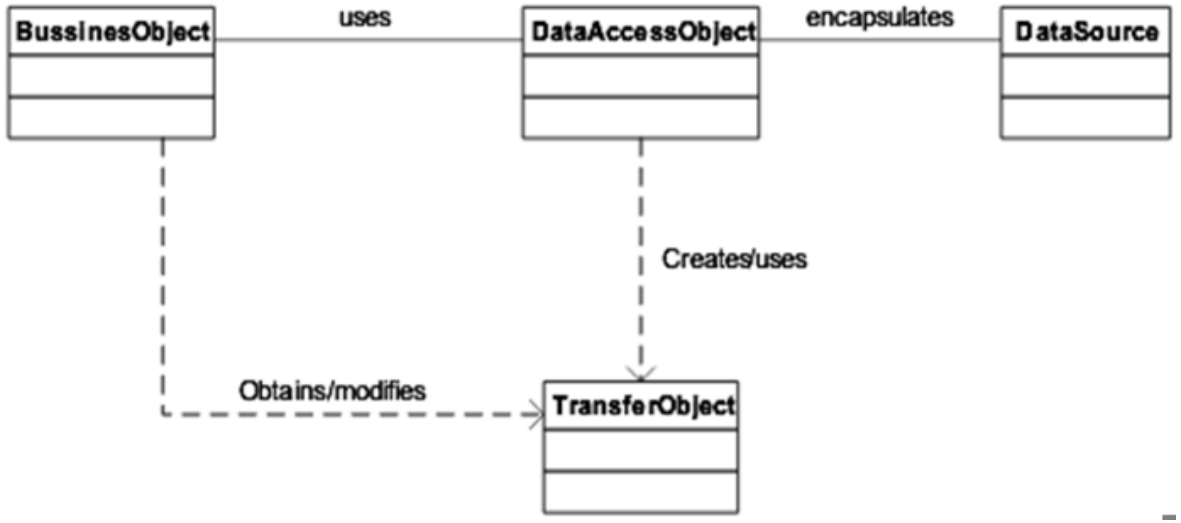




Facultad de
Ingeniería

Universidad de
La Sabana

Componentes:



- **BusinessObject**: Es el objeto que quiere acceder a la fuente de datos para poder almacenar o consultar datos.
- **DataAccessObject**: Abstrae al BusinessObject de los detalles del acceso a la fuente de datos.
- **DataSource**: Representa la implementación de la fuente de datos en sí.
- **TransferObject**: Es un objeto intermedio entre el BussinessObject y el DataAccessObject.

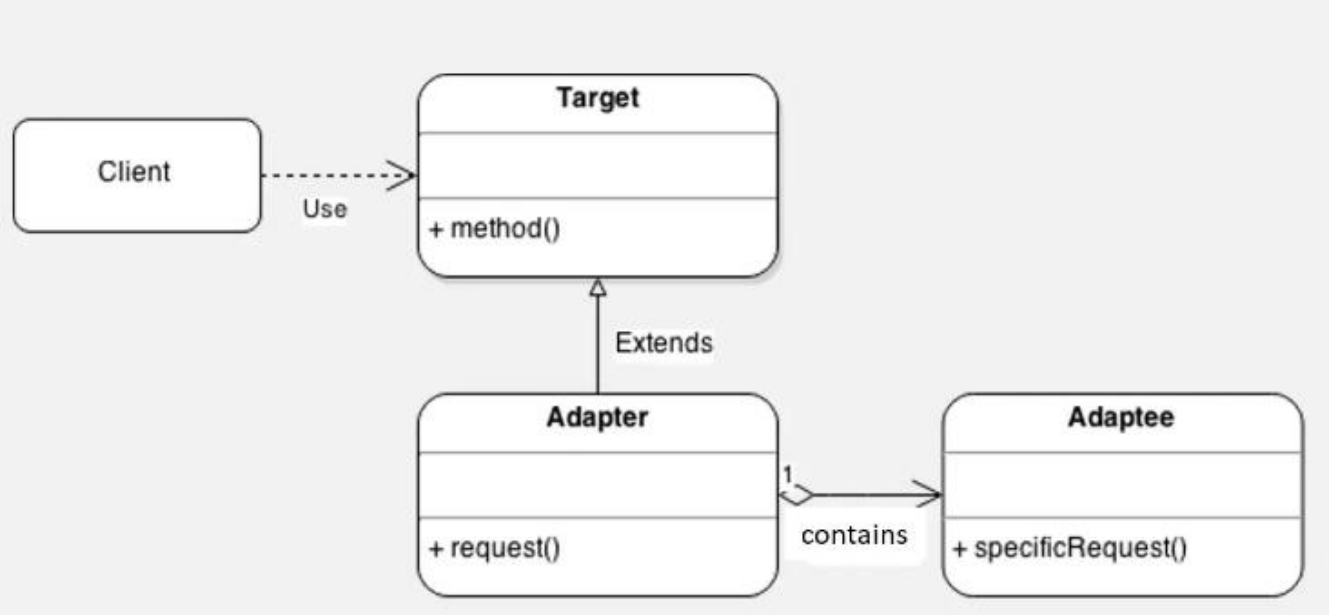


Patrones estructurales:

Adaptador

Definición:

El patrón adaptador es un patrón de diseño estructural que permite que las interfaces de una clase existente sean utilizadas como otra interfaz. Proporciona una envoltura alrededor de una interfaz existente para que sea compatible con otra interfaz sin modificar su código.





Facultad de
Ingeniería

Universidad de
La Sabana

Objetivo de Adaptador:

1. Permitir que interfaces incompatibles trabajen juntas.
2. Facilitar la reutilización de clases existentes.
3. Proporcionar una interfaz uniforme para diferentes sistemas.



Ejemplo



Client



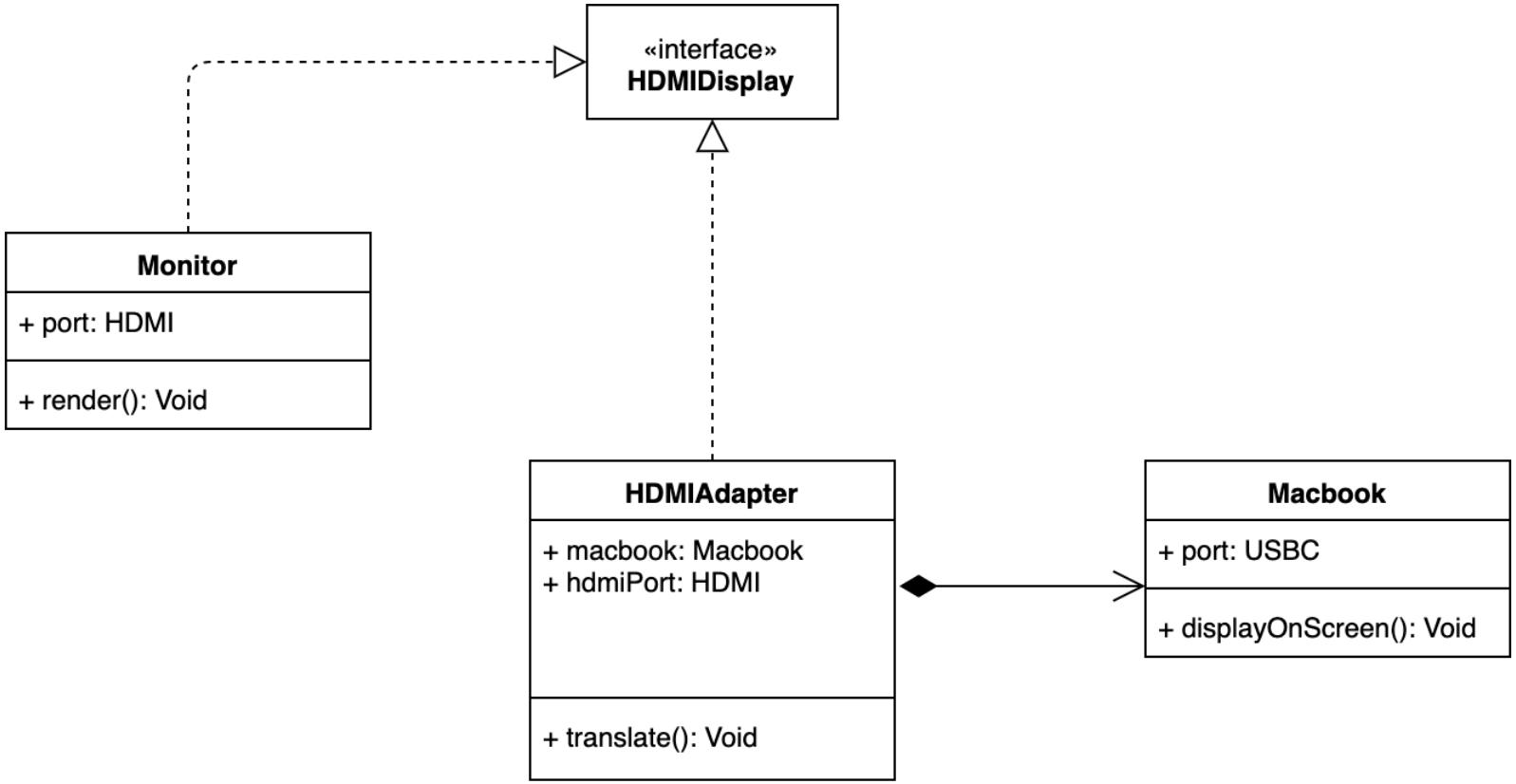
Adapter



Adaptee



Ejemplo


«interface»
HDMIDisplay
Monitor
+ port: HDMI
+ render(): Void
HDMIAdapter
+ macbook: Macbook
+ hdmiPort: HDMI
+ translate(): Void
Macbook
+ port: USBC
+ displayOnScreen(): Void


Lobby Ad Portas

Ventajas de Adaptador :



Reutilización de Código

Mantenimiento Sencillo

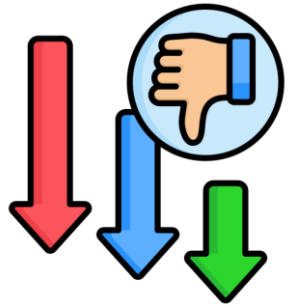
Interoperabilidad



Desventajas de Adaptador :

Complejidad Adicional

Posible Sobrecarga





Uso de un Adaptador:

```
// Cliente que utiliza la interfaz Target
public class Client {
        public static void main(String[] args) {
                // Utilizando el adaptador para integrar LegacySystem
                con la interfaz Target
                LegacySystem legacySystem = new LegacySystem();
                Target adaptedSystem = new Adapter(legacySystem);

                // Cliente interactúa con la interfaz Target
                adaptedSystem.request();
        }
}
```

Ejemplo de Implementación en Java:

```
// Interfaz Target
interface Target {
        void request();
}

// Clase existente con interfaz incompatible (Adaptee)
class LegacySystem {
        void specificRequest() {
                System.out.println("Legacy System Specific Request");
        }
}

// Adaptador que hace que LegacySystem sea compatible con Target
class Adapter implements Target {

        private final LegacySystem legacySystem;

        Adapter(LegacySystem legacySystem) {
                this.legacySystem = legacySystem;
        }

        @Override
        public void request() {
                legacySystem.specificRequest();
        }
}
```

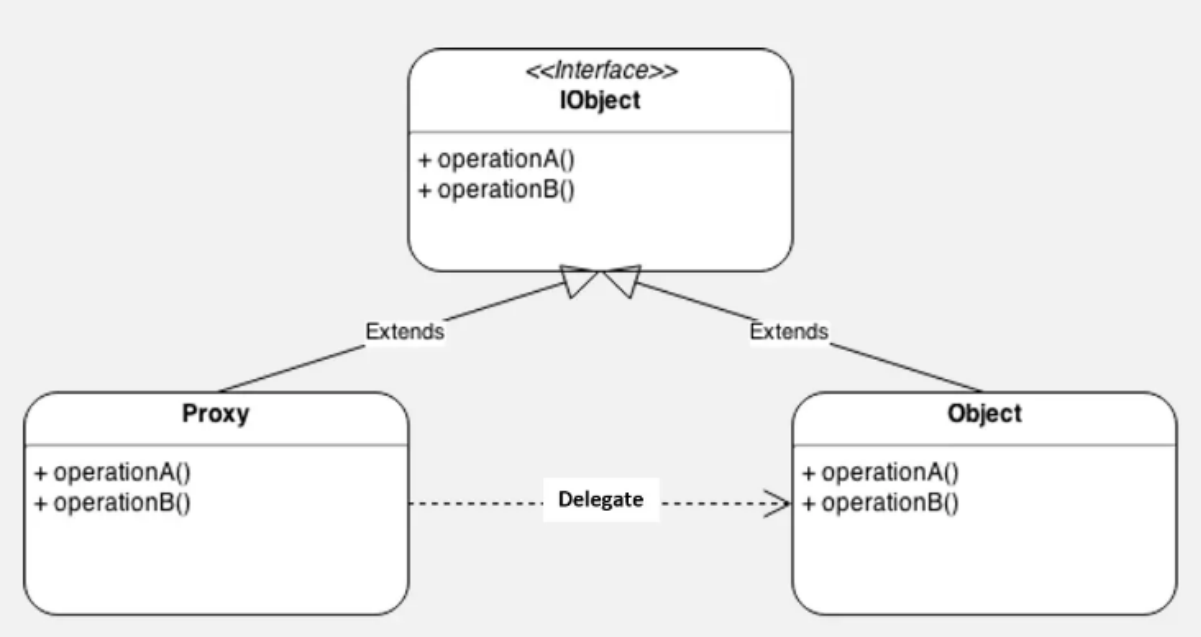



Patrones estructurales:

Proxy

Definición:

El patrón Proxy es un patrón de diseño estructural que proporciona un objeto representativo (proxy) que controla el acceso a otro objeto (sujeto). El proxy actúa como un intermediario, permitiendo agregar funcionalidades adicionales, como control de acceso, registro, o carga diferida, sin modificar la lógica del sujeto real.





Facultad de
Ingeniería

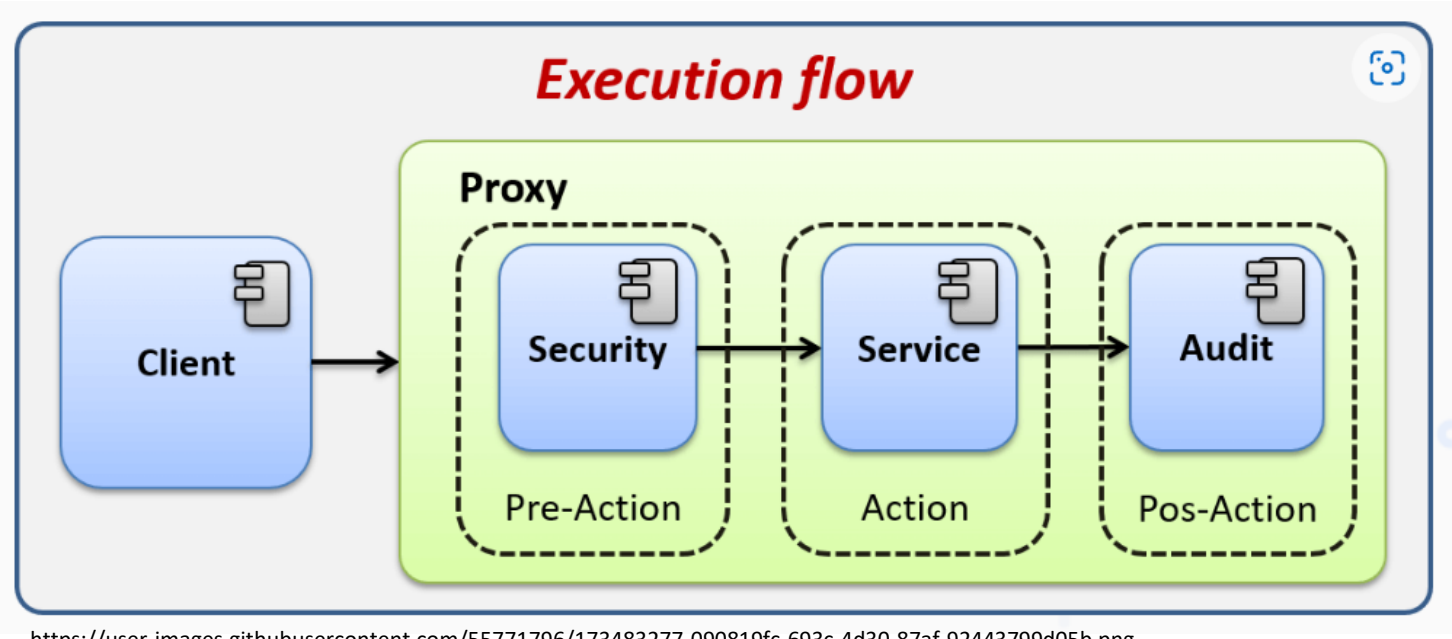
Universidad de
La Sabana

Objetivo de Proxy:

1. Controlar el acceso a un objeto mediante un intermediario.
2. Proporcionar un nivel adicional de abstracción y control.
3. Permitir funcionalidades adicionales antes o después de la invocación del sujeto.



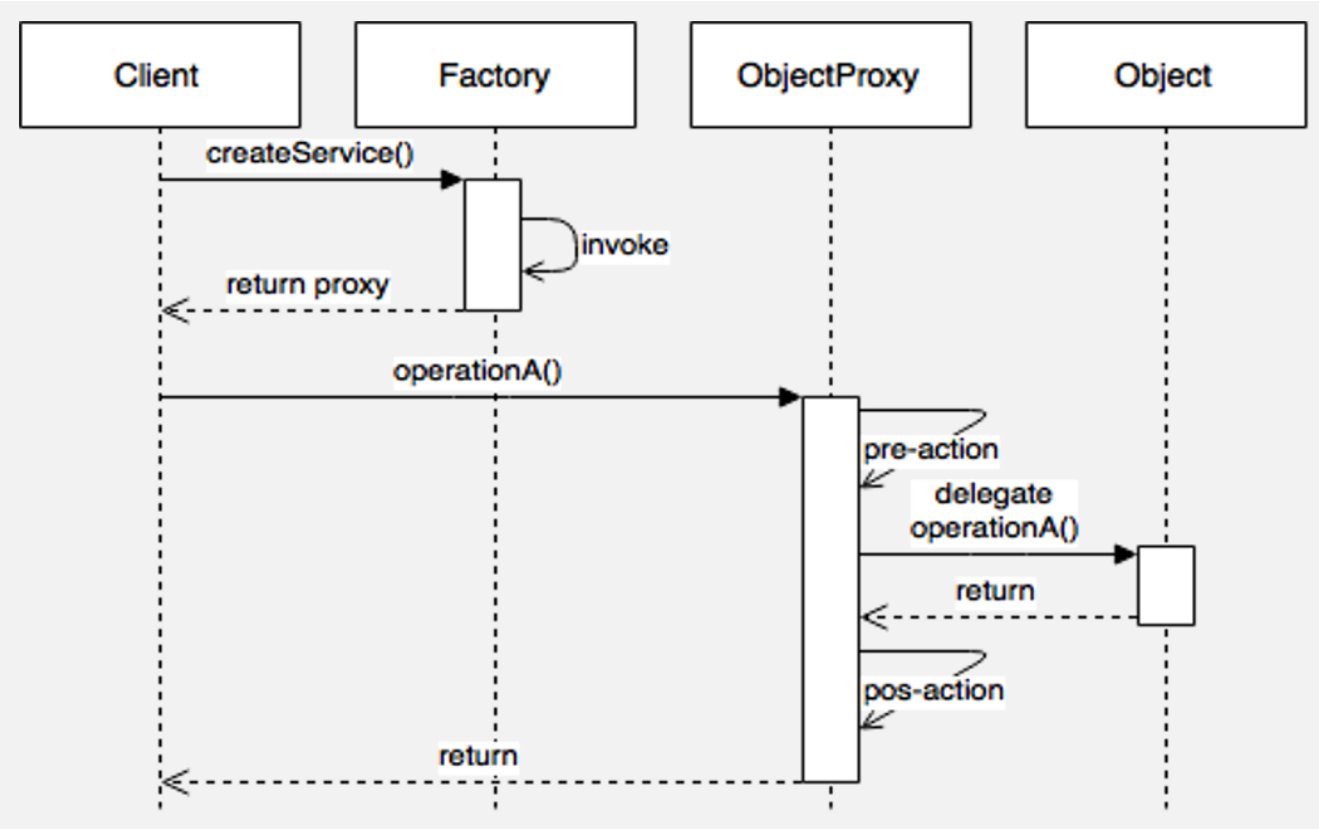
Ejemplo



https://user-images.githubusercontent.com/55771796/173483277-090819fc-693c-4d30-87af-92443799d05b.png



Flujo patrón proxy





Ventajas de Proxy :

Control de Acceso



Registro de Operaciones

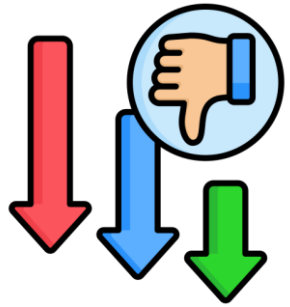
Carga Diferida



Desventajas de Proxy :

Complejidad Adicional

Sutilezas en la Implementación





Uso de un Proxy:

```
// Cliente que utiliza el Proxy
public class Client {
        public static void main(String[] args) {

                // Utilizando el Proxy para controlar el acceso a una imagen
                Image image = new ProxyImage("example.jpg");

                // La imagen real se carga y muestra solo cuando se llama a display()
                image.display();

                // En llamadas subsiguientes, el Proxy reutiliza la imagen ya cargada
                image.display();
        }
}
```

```
// Proxy: Controla el acceso a RealSubject (RealImage)
class ProxyImage implements Image {
        private RealImage realImage;
        private final String filename;

        ProxyImage(String filename) {
                this.filename = filename;
        }

        @Override
        public void display() {
                if (realImage == null) {
                        // Crear una instancia de RealImage solo cuando sea necesario
                        realImage = new RealImage(filename);
                }
                realImage.display();
        }
}
```

Ejemplo de Implementación en Java:

```
// Interfaz Image
interface Image {
        void display();
}

// Clase RealSubject: Implementación concreta de Image
class RealImage implements Image {
        private final String filename;

        RealImage(String filename) {
                this.filename = filename;
                loadImageFromDisk();
        }

        private void loadImageFromDisk() {
                System.out.println("Loading image: " + filename);
        }

        @Override
        public void display() {
                System.out.println("Displaying image: " + filename);
        }
}
```



Facultad de
Ingeniería

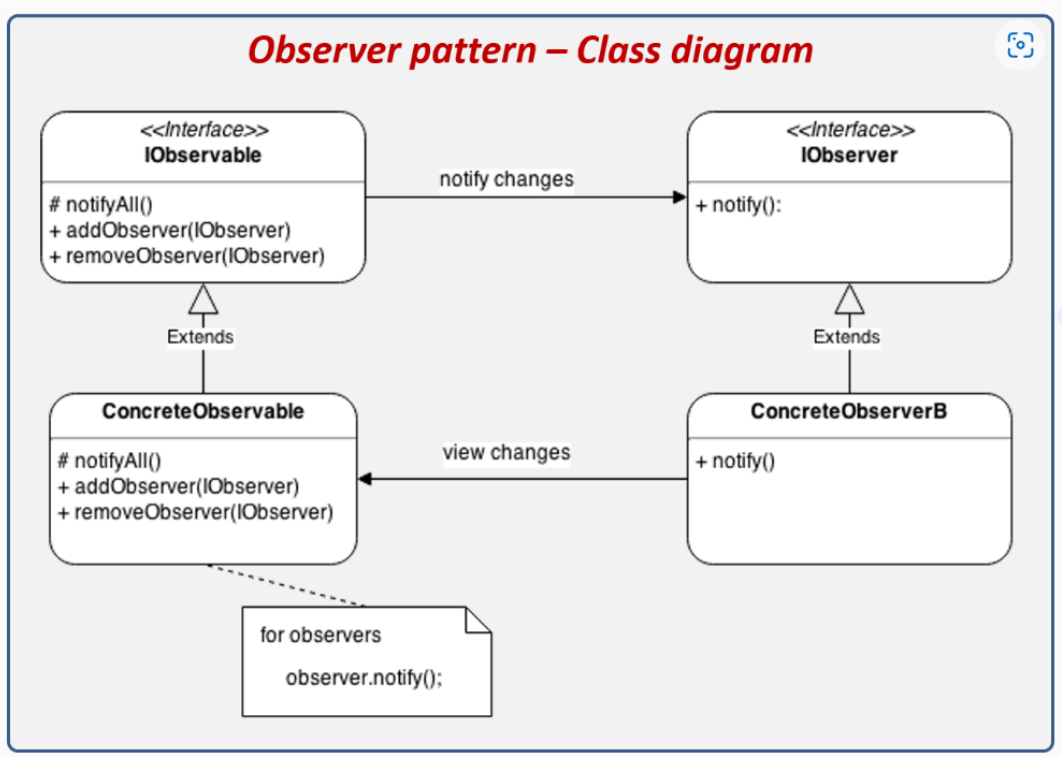
Universidad de
La Sabana

Patrones de comportamiento:

Observer

Definición:

Define una dependencia de un único objeto a muchos otros de tal manera que cuando el único objeto cambia su estado, todos los objetos dependientes son notificados y actualizados automáticamente.

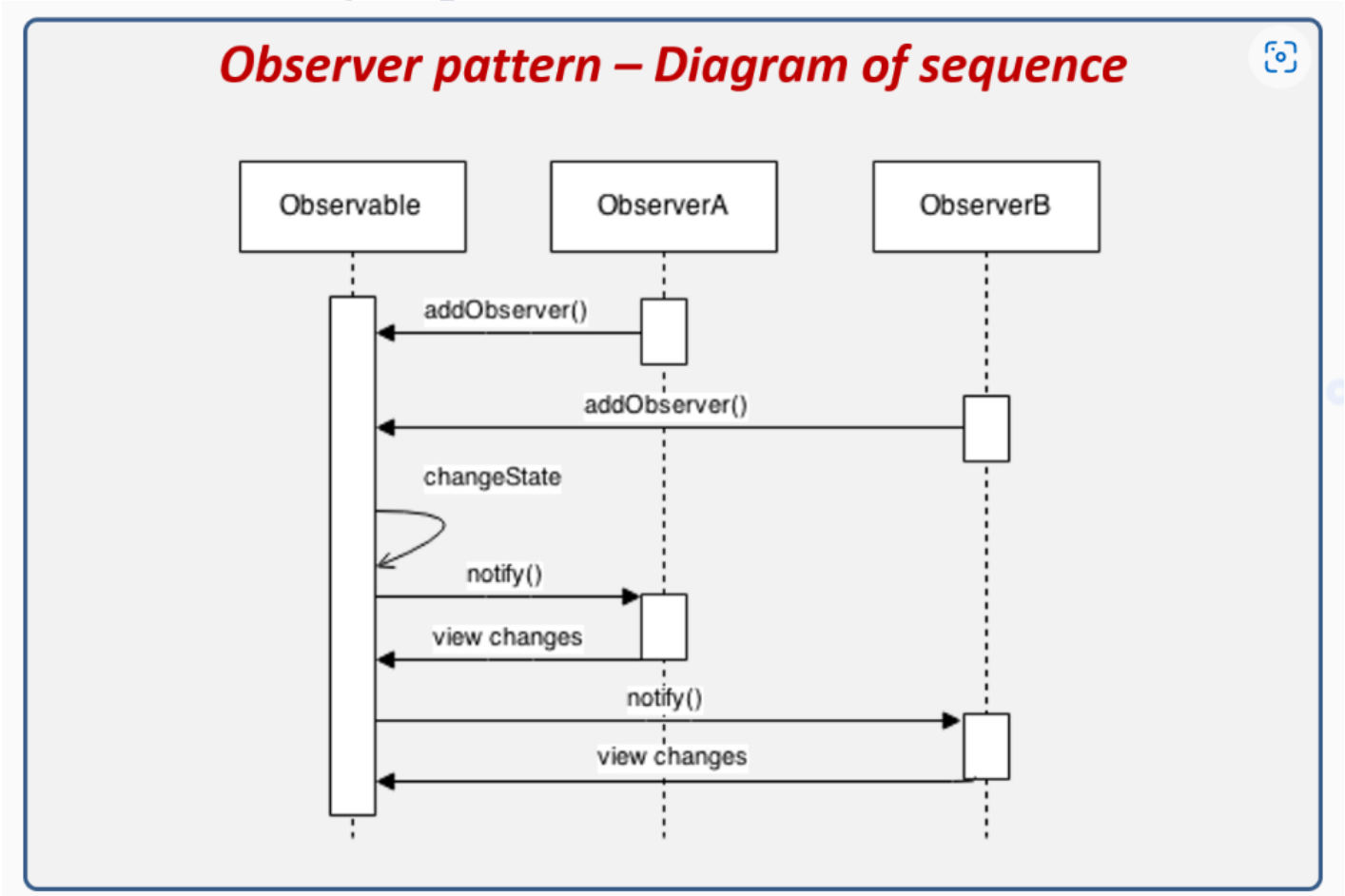




Facultad de **Ingeniería**

Universidad de La Sabana

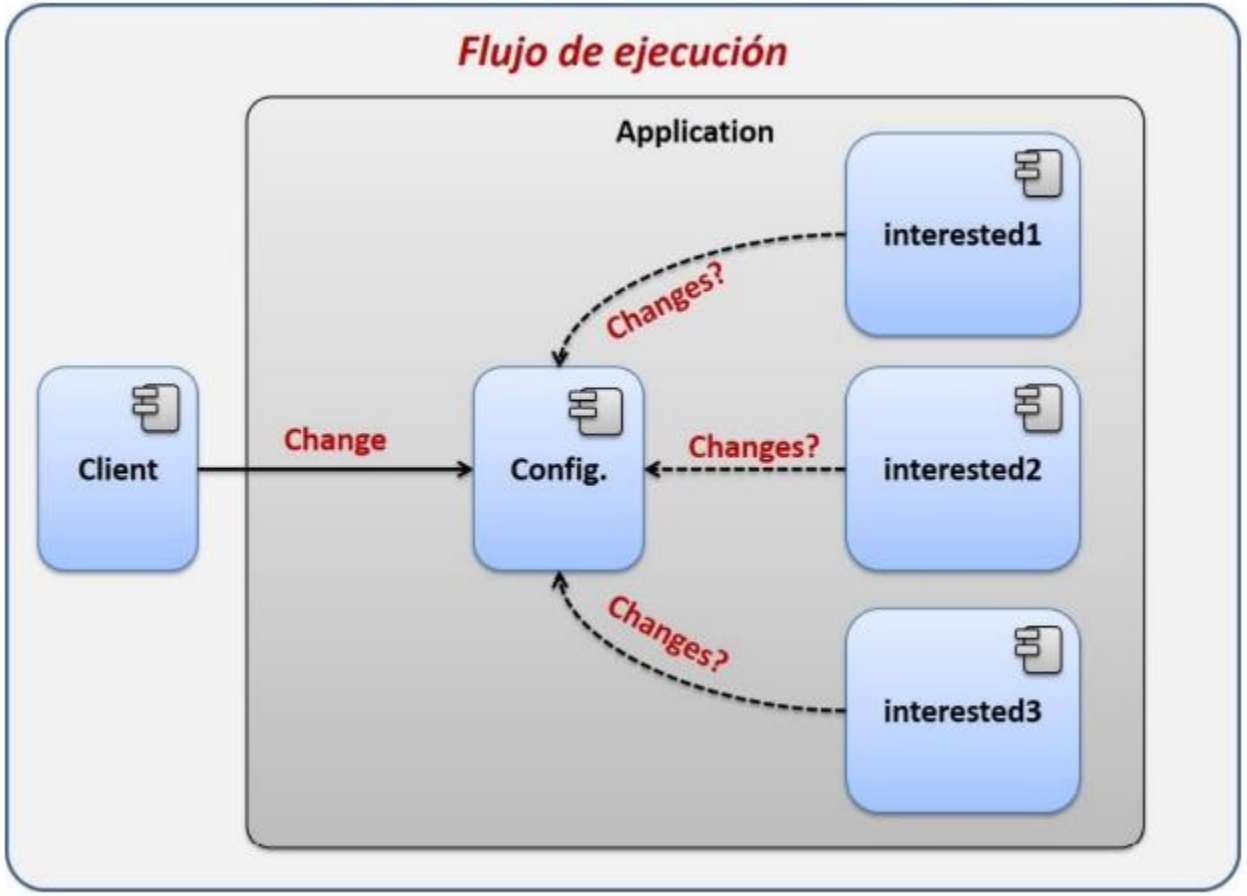
Flujo patrón Observer





Ejemplo

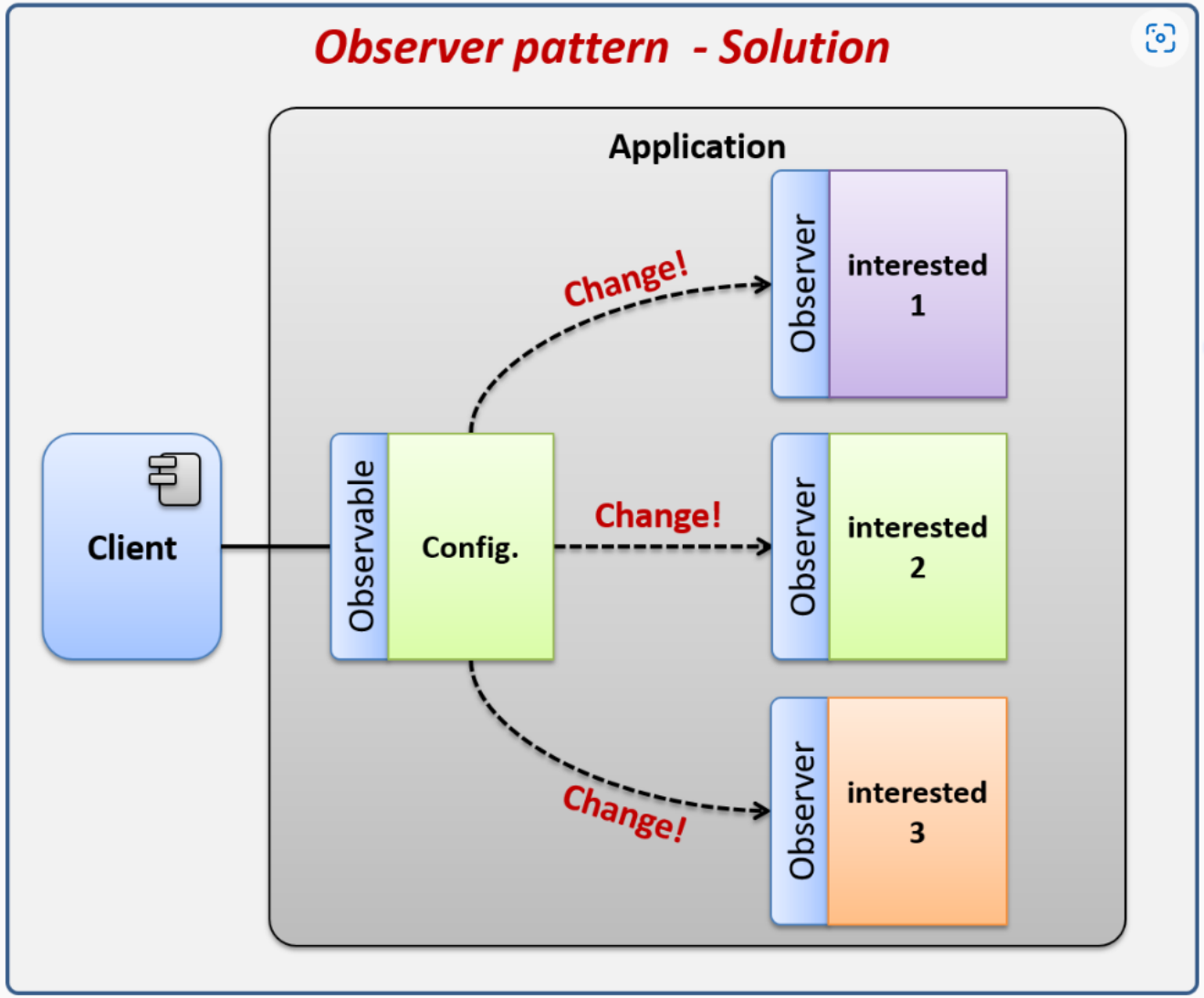
Mediante la implementación del patrón de diseño Observer desarrollaremos una aplicación que cargue desde el inicio la configuración del sistema. Dicha configuración notificará de forma automática a todos los objetos que estén interesados en los cambios, todo esto con la finalidad de que los objetos interesados en la configuración no tengan que consultar cada momento si existe algún cambio importante para reflejarlo al usuario.





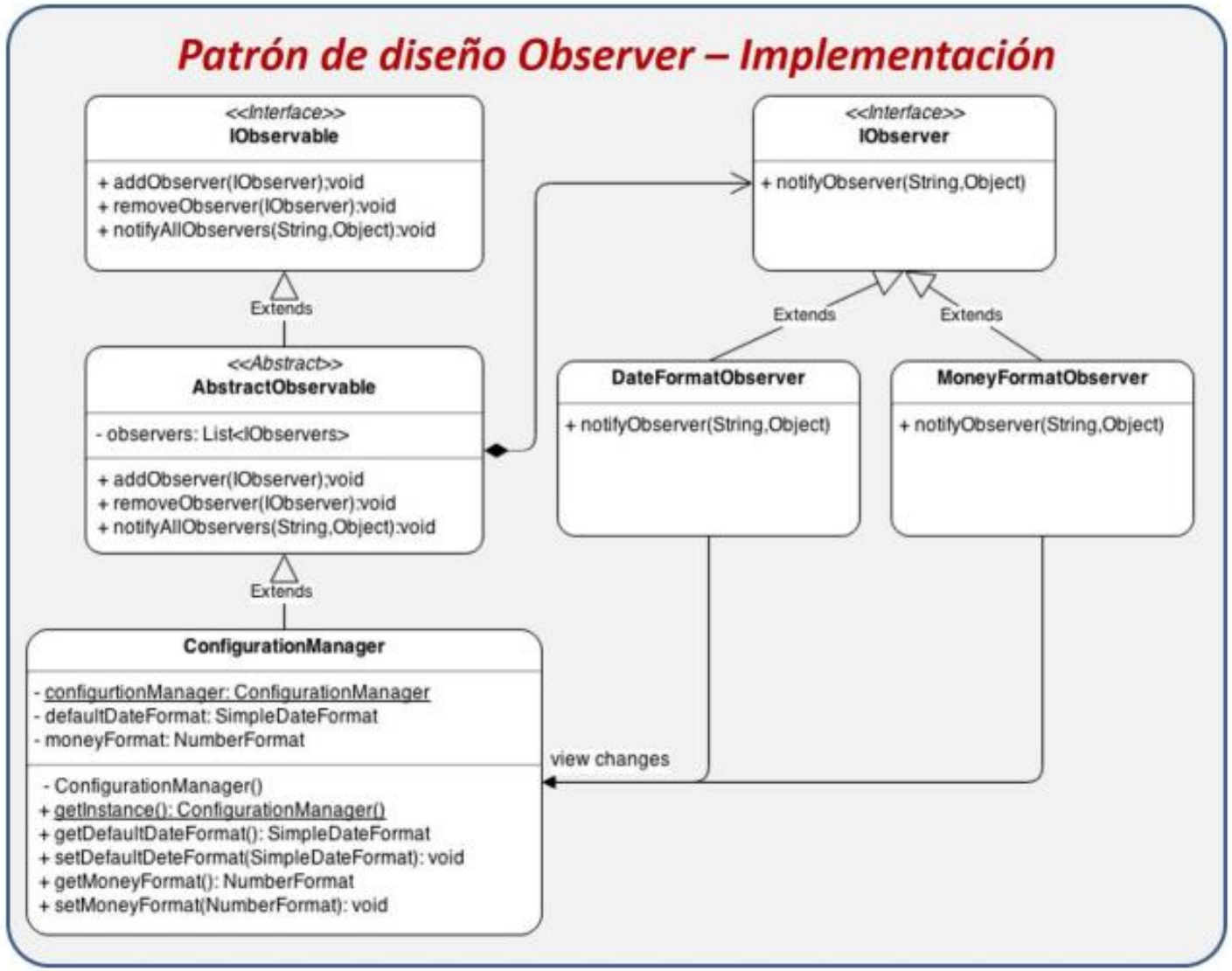
Solución

Solución, con el patrón Observer:





Solución Ejercicio



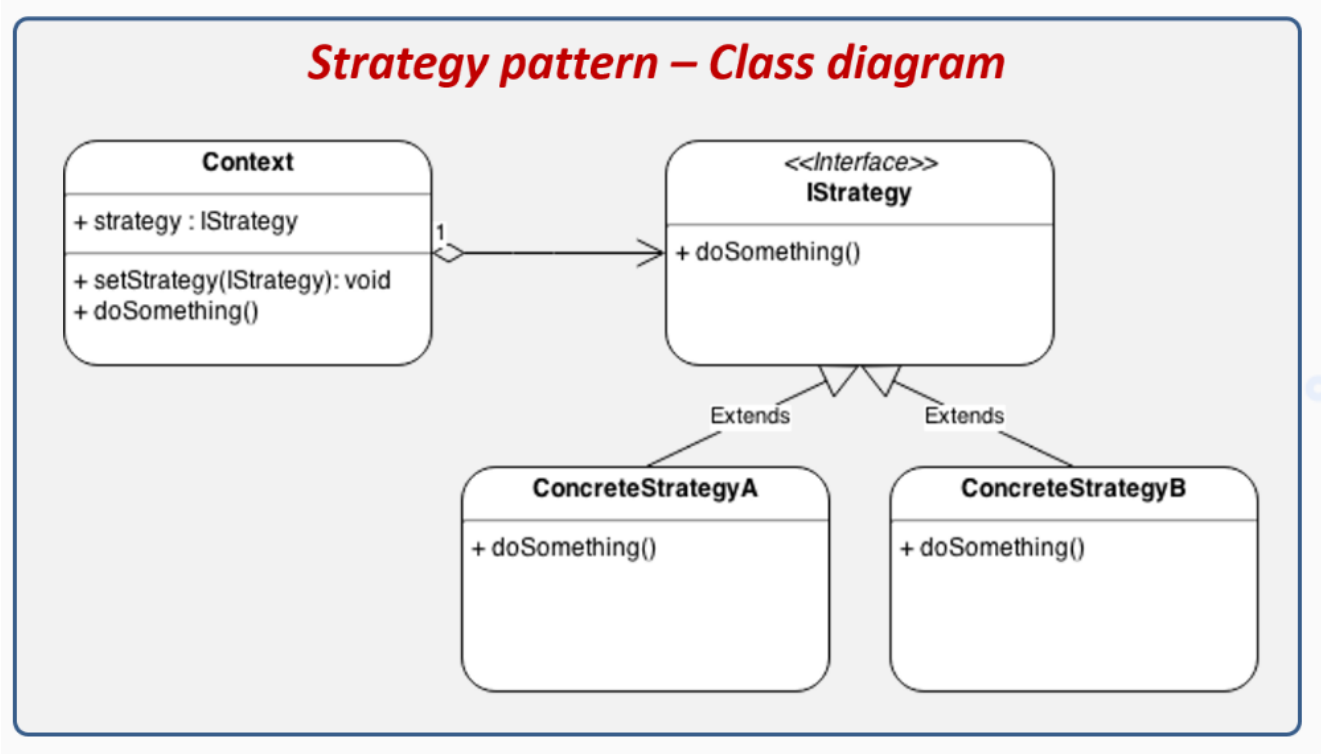


Patrones de comportamiento:

Strategy

Definición:

Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Este patrón permite que los algoritmos varíen de forma independiente a los clientes que los usen.

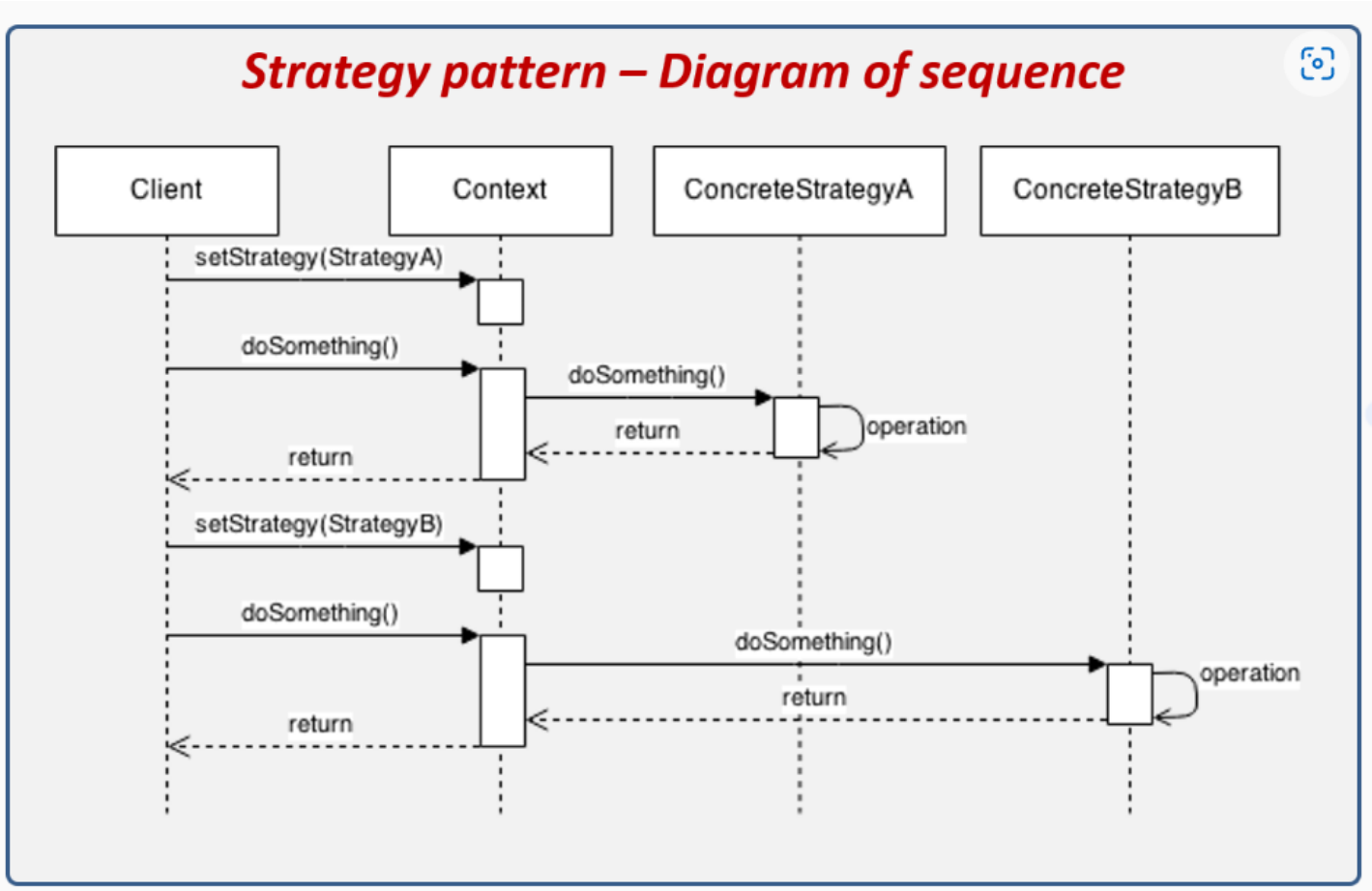




Facultad de **Ingeniería**

Universidad de La Sabana

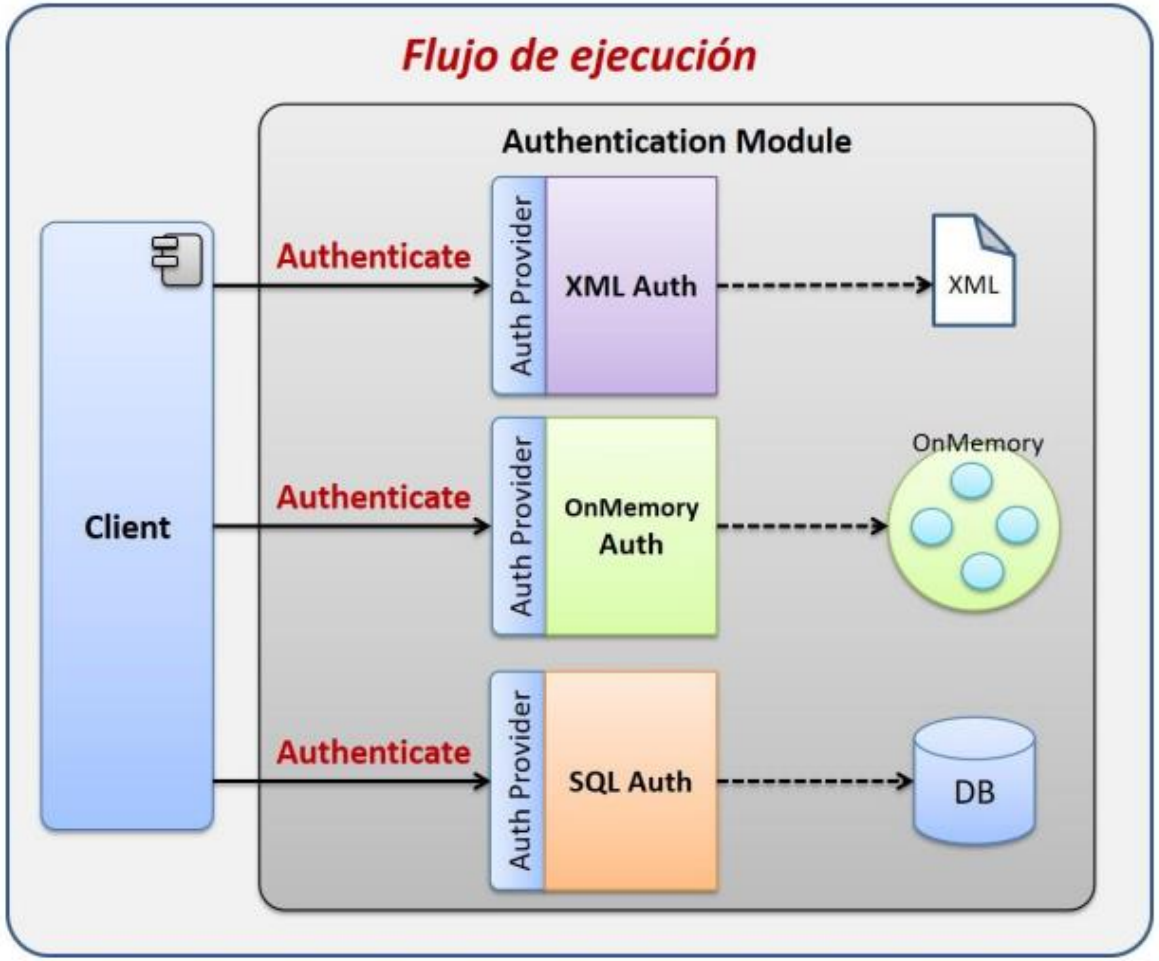
Flujo patrón Strategy





Ejemplo

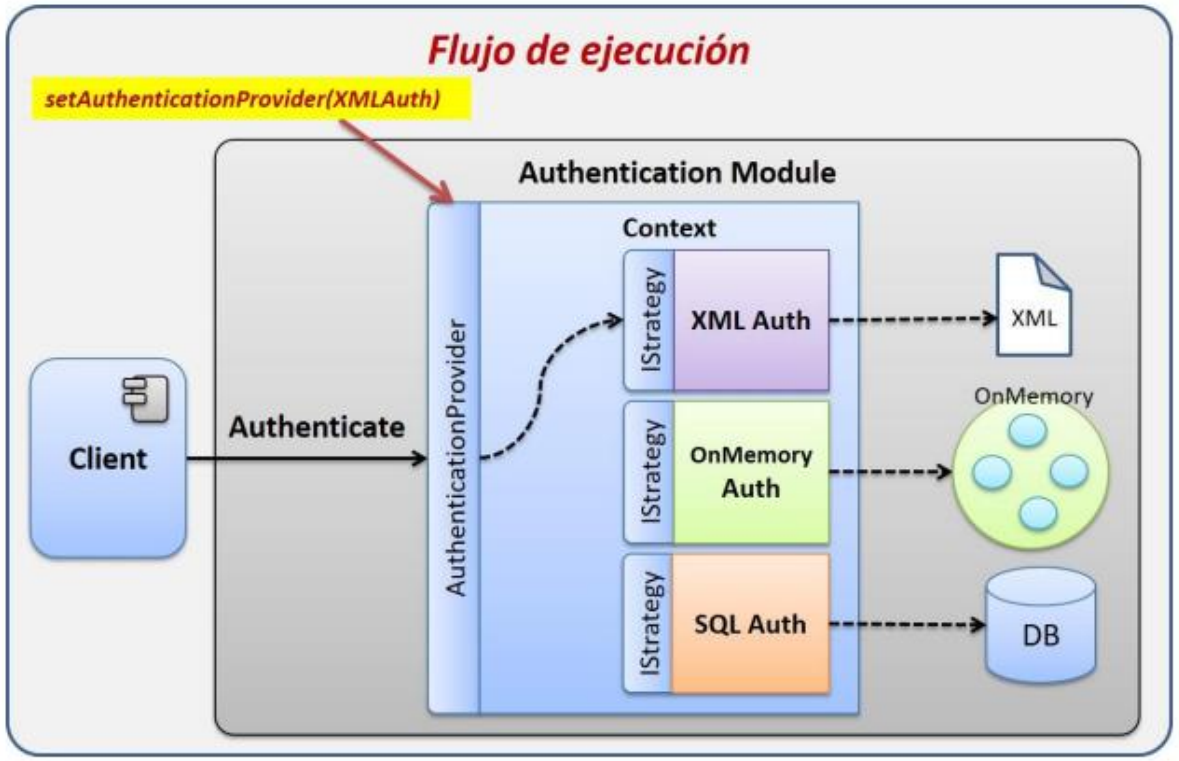
Mediante la implementación del patrón de diseño Strategy desarrollaremos una aplicación que permita la autenticación mediante diversos métodos. El usuario podrá autenticarse mediante una configuración de usuario por XML, Base de datos o una configuración en memoria. Mediante el patrón Strategy, nuestro cliente podrá configurar la aplicación para elegir que método de autenticación le es más conveniente, todo esto, sin necesidad de programar nada adicional.

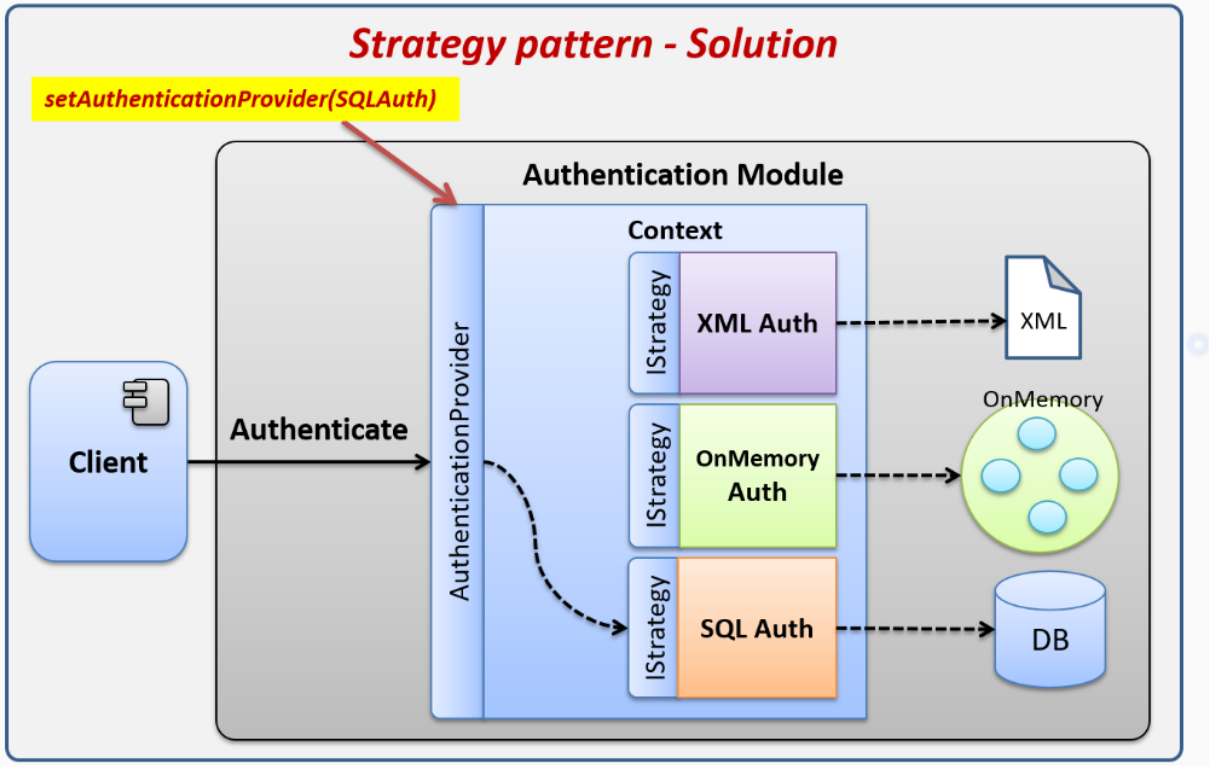




Solución

Solución, con el patrón Strategy:







Solución Ejercicio





Facultad de
Ingeniería



Universidad de
La Sabana